



life.augmented



STM32CubeMonitor

Performance monitor
dashboard
Hands on

Purpose

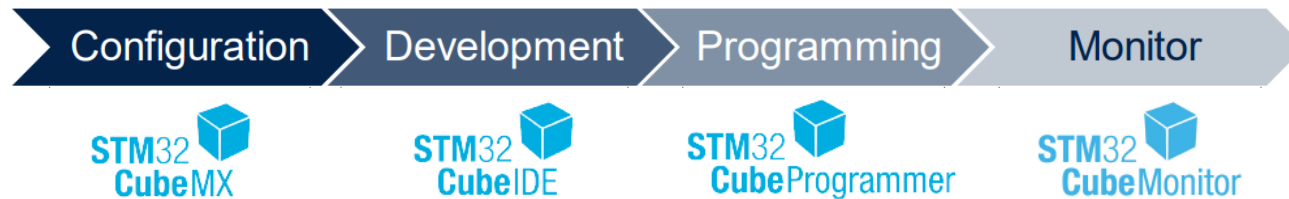
- Have an overview of STM32CubeMonitor: what it is and how to monitor the target application running on the STM32H735-DK board.
- Have an overview of the customization of the dashboard.
- Discover the dashboard used in next HandsOn2 and HandsOn3 and make it ready.

Presentation of the tool



Presentation of the tool

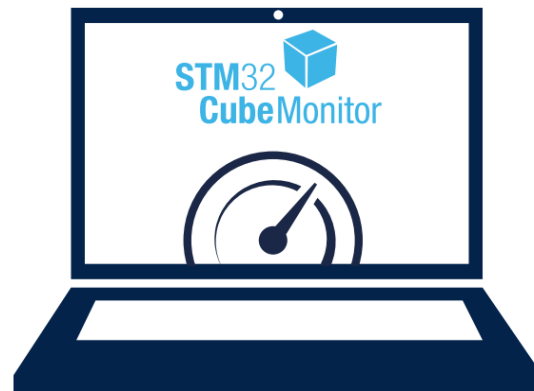
- STM32CubeMonitor is part of STM32Cube software tools
 - help developers **fine-tune** the behavior and performance of their applications in **real time**.



- **Free download** from [st.com](https://www.st.com)
- Based on an open platform used by a large community of people, not only in electronics industry.
 - Node-RED®
 - ST has added features, especially for the communication with STM32 devices.

Presentation of the tool

- STM32CubeMonitor is a software tool to monitor and visualize real-time data from STM32 devices **while the application is running**.
 - Allows users **customize the monitoring** of their STM32 targets according to their needs.
 - Allows to quickly build **custom dashboards** with various data visualization **widgets** and charts.
- graphical **flow-based editor** for visual programming



Real-time variable monitoring
during application runtime

Flexible graphical visualization

Direct support of the Node-RED®
open community

Presentation of the tool

- STM32CubeMonitor performs data acquisition by reading in the target MCU memory through ST-Link and SWD/JTAG connection.
- The elements involved are :
 - STM32 MCU debug block
 - which performs access to the memory through the MCU bus.
 - STM32 JTAG/SWD access port
 - used to connect the MCU debug block to ST-Link.
 - ST-Link device.
 - USB protocol on the computer.
 - STM32CubeMonitor software.
- **All the MCU address** space is accessible.

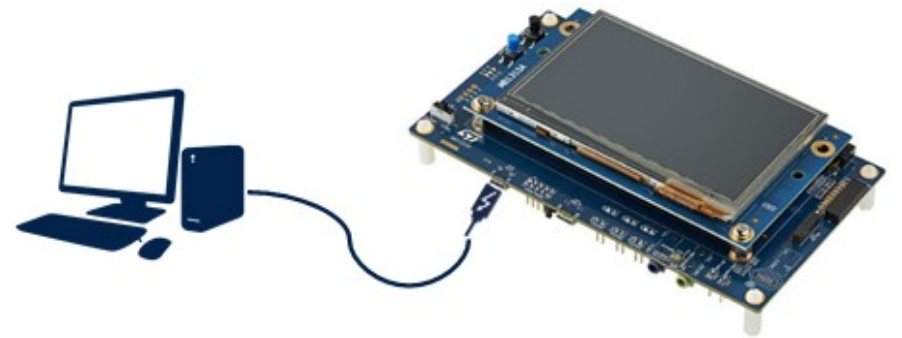


Let's start with STM32CubeMonitor



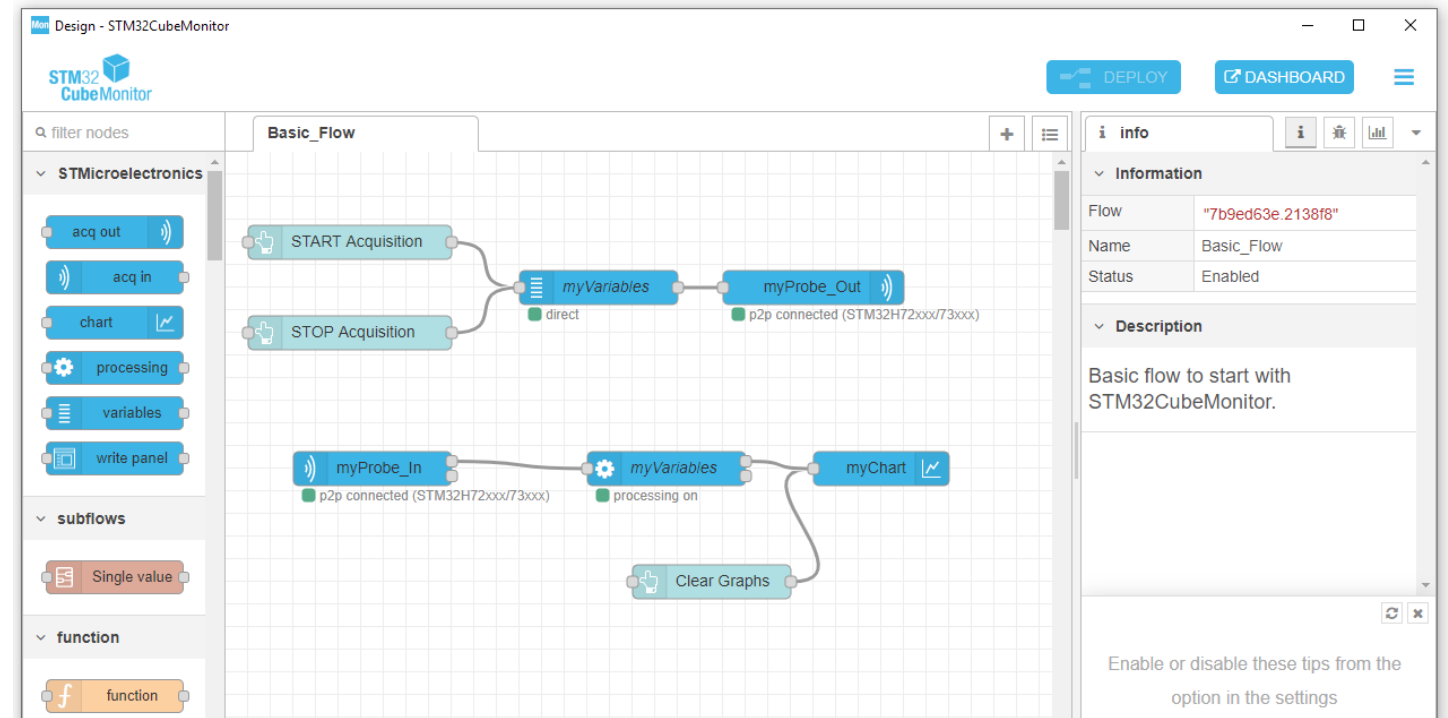
Restarting from homework configuration

- During homework you have loaded a firmware to the STM32H735-DK board.
 - It will be used during this HandsOn1
- You have started the STM32CubeMonitor with the default basic flow and make the connection of the tool to the STM32H735-DK board.
- Let's restart from this point
 - Please plug in your STM32H735-DK board to your PC
 - Launch STM32CubeMonitor 



The design mode

- STM32CubeMonitor starts in design mode
 - The basic flow is displayed
- The connection to the STM32H735-DK board is displayed
 - Automatically done



Flow structure

- One TX path from PC to probe

- START Acquisition

Messages are sent on the ST-Link:

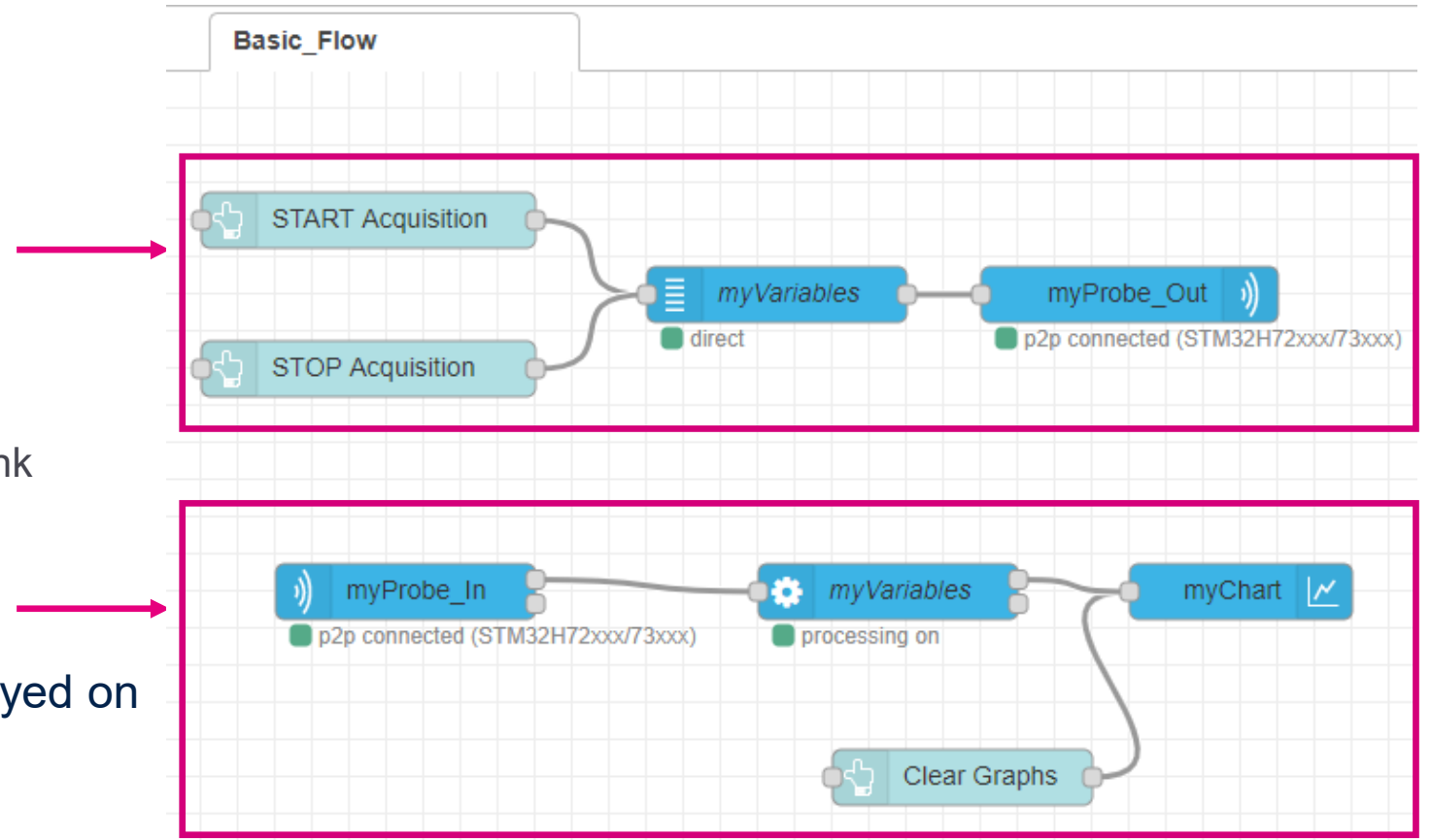
- "Read sequentially myVariables"

- STOP Acquisition

No more messages are sent on the ST-Link

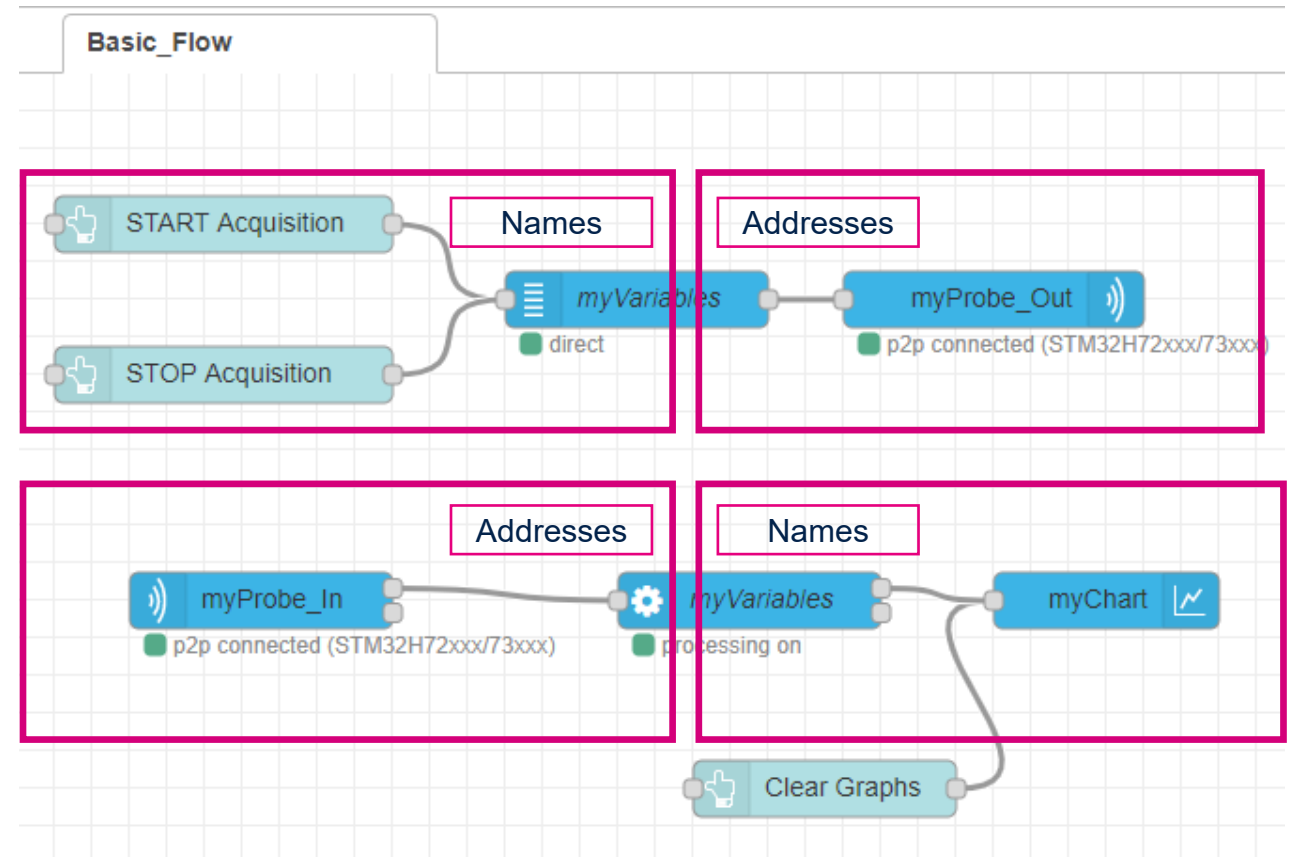
- One RX path from probe to PC

- MyVariables are received and displayed on the chart.



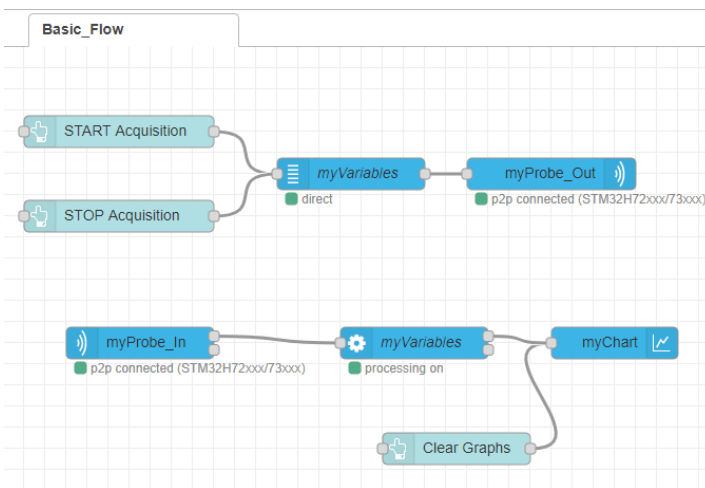
Name and Address

- The MCU and its MCU debug block use addresses
- Humans prefer names
- PC software also!
 - The nodes “myVariables” manages the translation



The dashboard mode

- The widgets seen on the flow are displayed on the dashboard.
 - 3 Buttons
 - START, STOP, CLEAR
 - Chart



Read variables from MCU RAM memory

STM32 
CubeMonitor

Variable name

- Let's access to some variable of the HandsOn1 firmware

- The code defines the type of "basicCounter"

- Unsigned 16-bit

```
70  
71 uint16_t basicCounter = 0;  
72
```

- The code increment the "basicCounter" every 500ms

```
134  
135 /* Infinite loop */  
136 /* USER CODE BEGIN WHILE */  
137 while (1)  
138 {  
139     /* USER CODE END WHILE */  
140  
141     /* USER CODE BEGIN 3 */  
142     HAL_Delay(500);  
143     HAL_GPIO_TogglePin(GPIOC, USER_LED1_Pin);  
144     basicCounter += 1;  
145
```

Get the names from elf

- The variable names are in the elf executable file

- generated during the project build in STM32CubeIDE
- STM32H735-DK.elf



```
arm-none-eabi-gcc -o "STM32H735G-DK.elf" @"objects.list" -mcpu=cortex-m7 -T"C:\STM32H735G-DK\STM32H735G-DK.ld"
Finished building target: STM32H735G-DK.elf

arm-none-eabi-size STM32H735G-DK.elf
arm-none-eabi-objdump -h -S STM32H735G-DK.elf > "STM32H735G-DK.list"
arm-none-eabi-objcopy -O binary STM32H735G-DK.elf "STM32H735G-DK.bin"
  text    data    bss     dec     hex filename
 7692     40    1568    9300    2454 STM32H735G-DK.elf
Finished building: default.size.stdout

Finished building: STM32H735G-DK.list

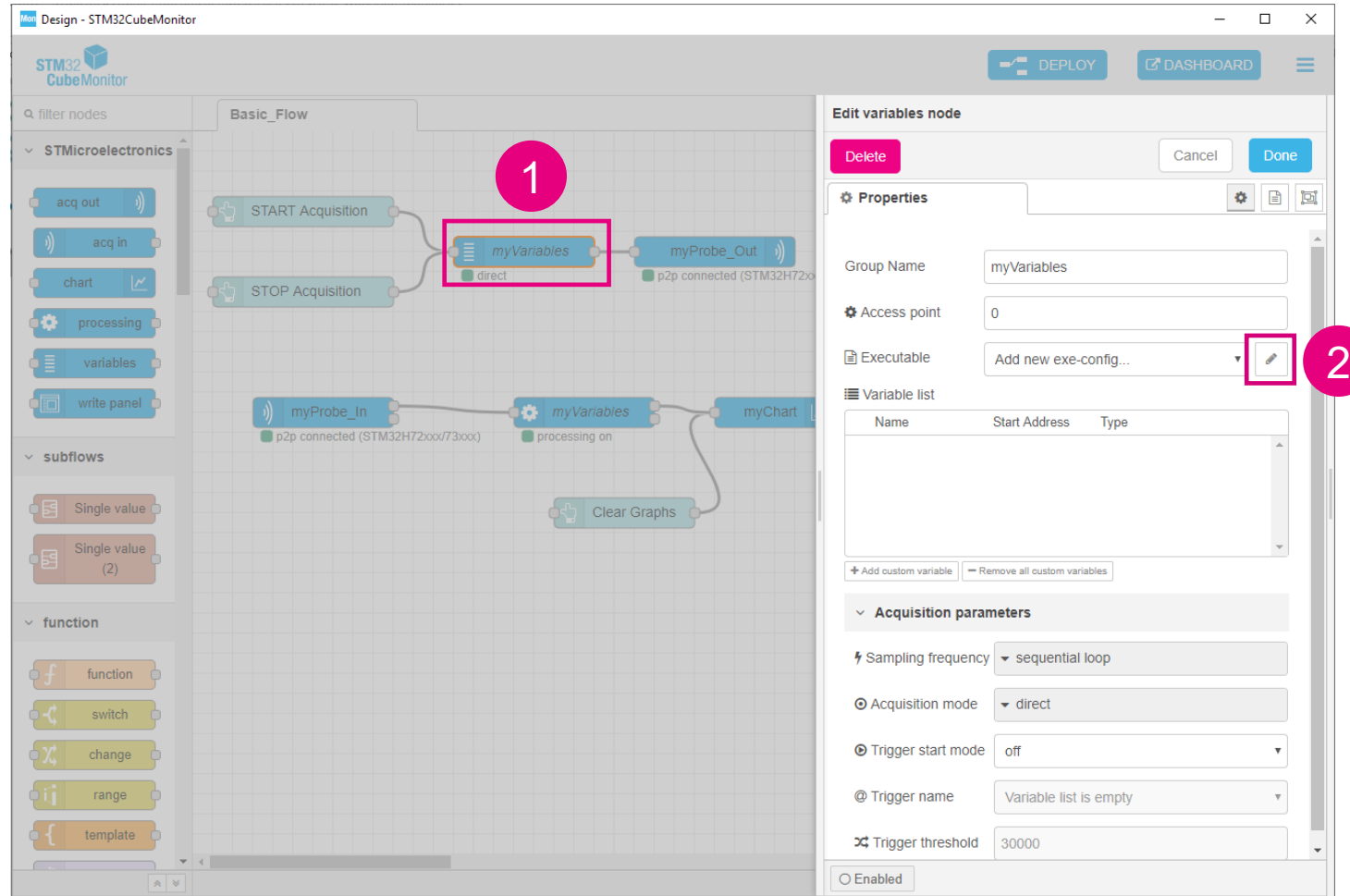
Finished building: STM32H735G-DK.bin

00:05:51 Build Finished. 0 errors, 2 warnings. (took 17s.365ms)
```

- .out and .axf extensions also supported

Get the names from elf

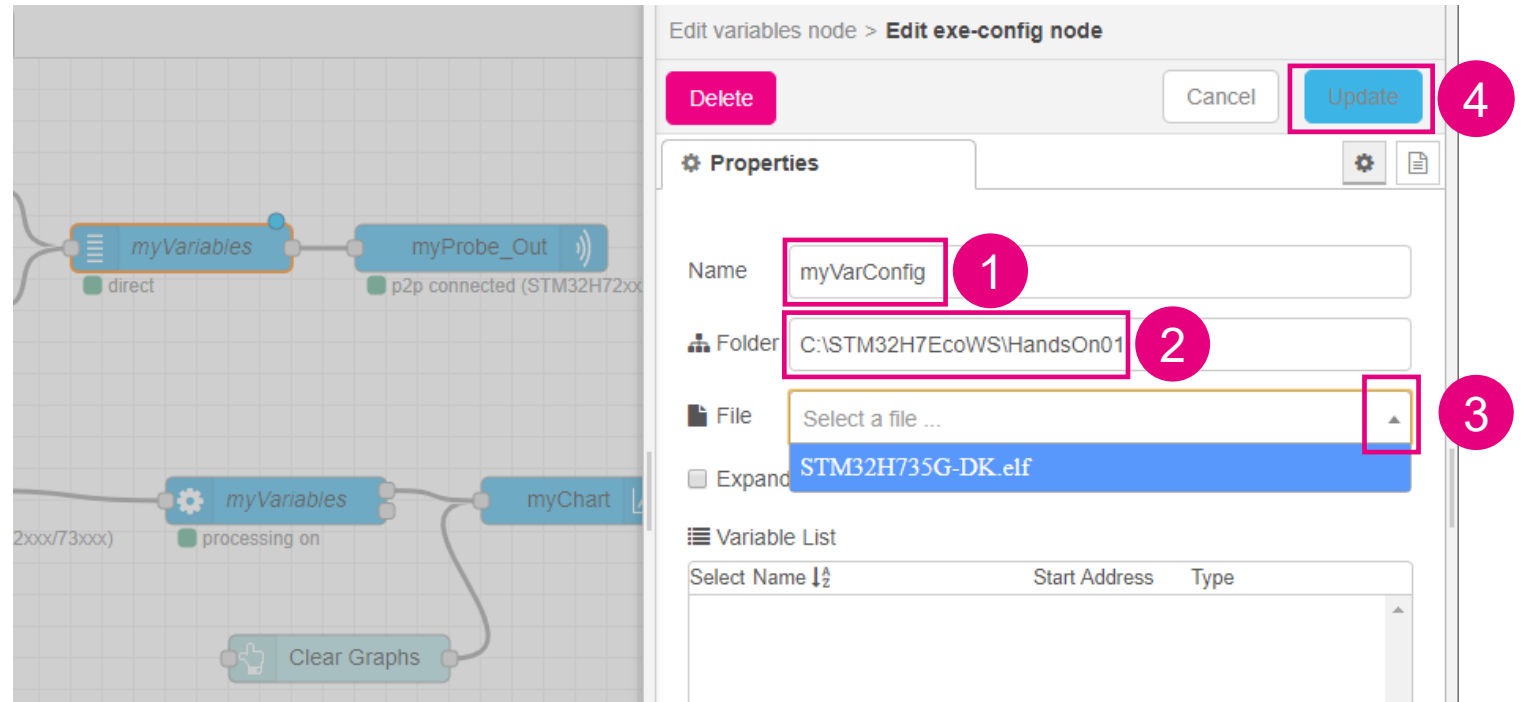
- Add new exe-config...
 1. Double-click on “myVariables”
 - The variables node properties are displayed
 2. Click on the pen of “Executable”



Get the names from elf

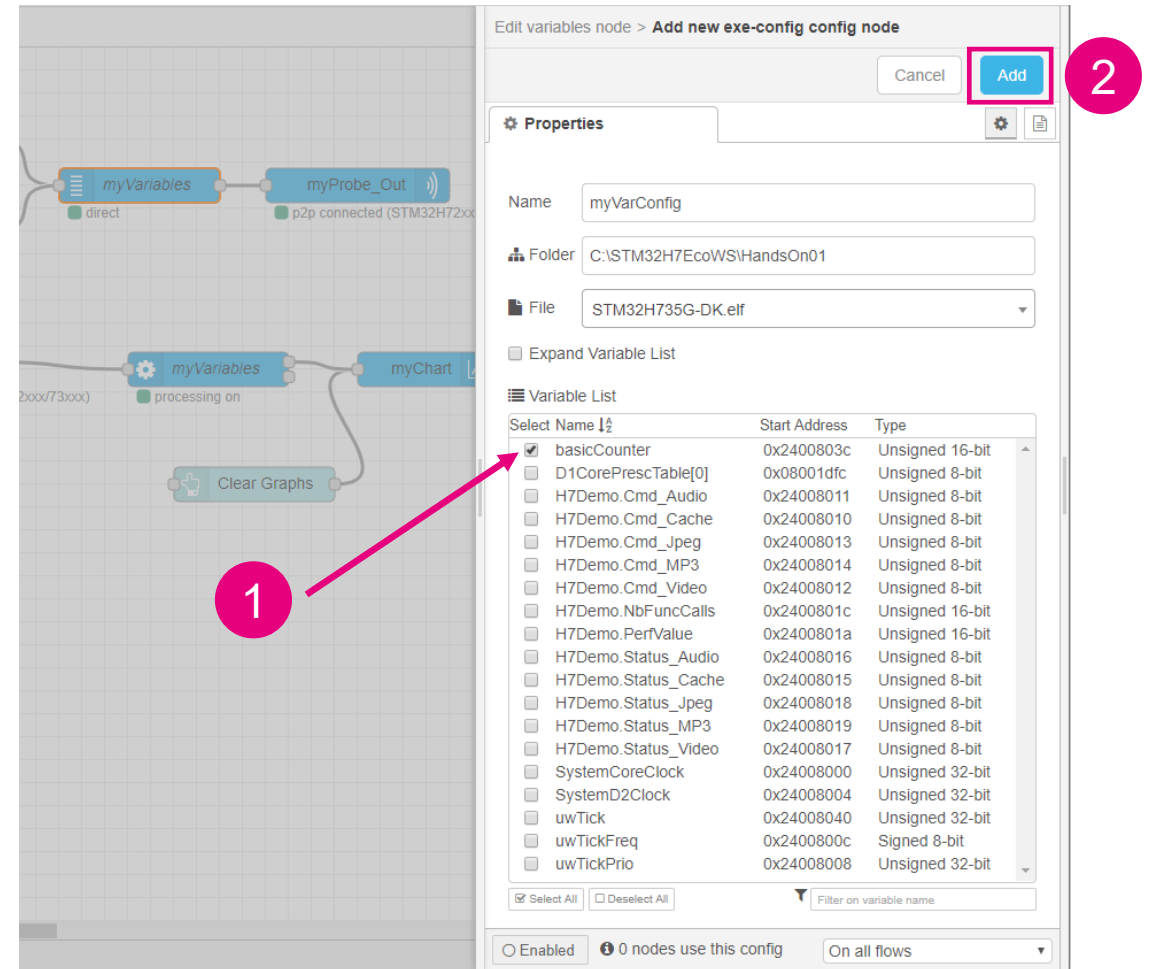
- Create the exe-config

1. Give a name to the configuration
 - C:\STM32H7EcoWS\HandsOn_01
2. Fill "Folder" with the folder containing the elf file
 - C:\STM32H7EcoWS\HandsOn_01
3. Select the elf file from the drop-down list
 - STM32H735G-DK.elf
4. Click in Update



Get the names from elf

- Select the variables in the “Variable List”
 1. Select “basicCounter”
 2. Click on “Add”
 - Adds the configuration to the “myVariables”

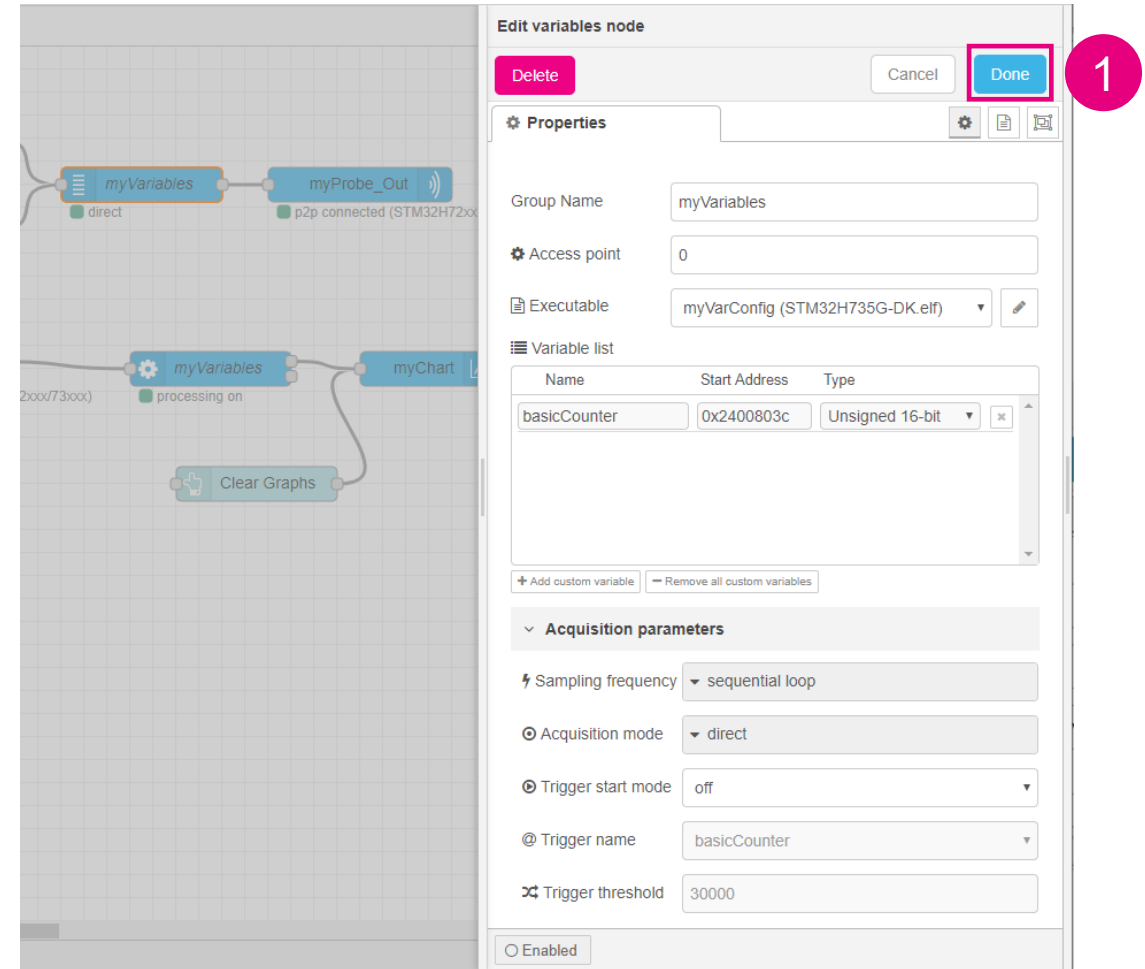


Get the names from elf

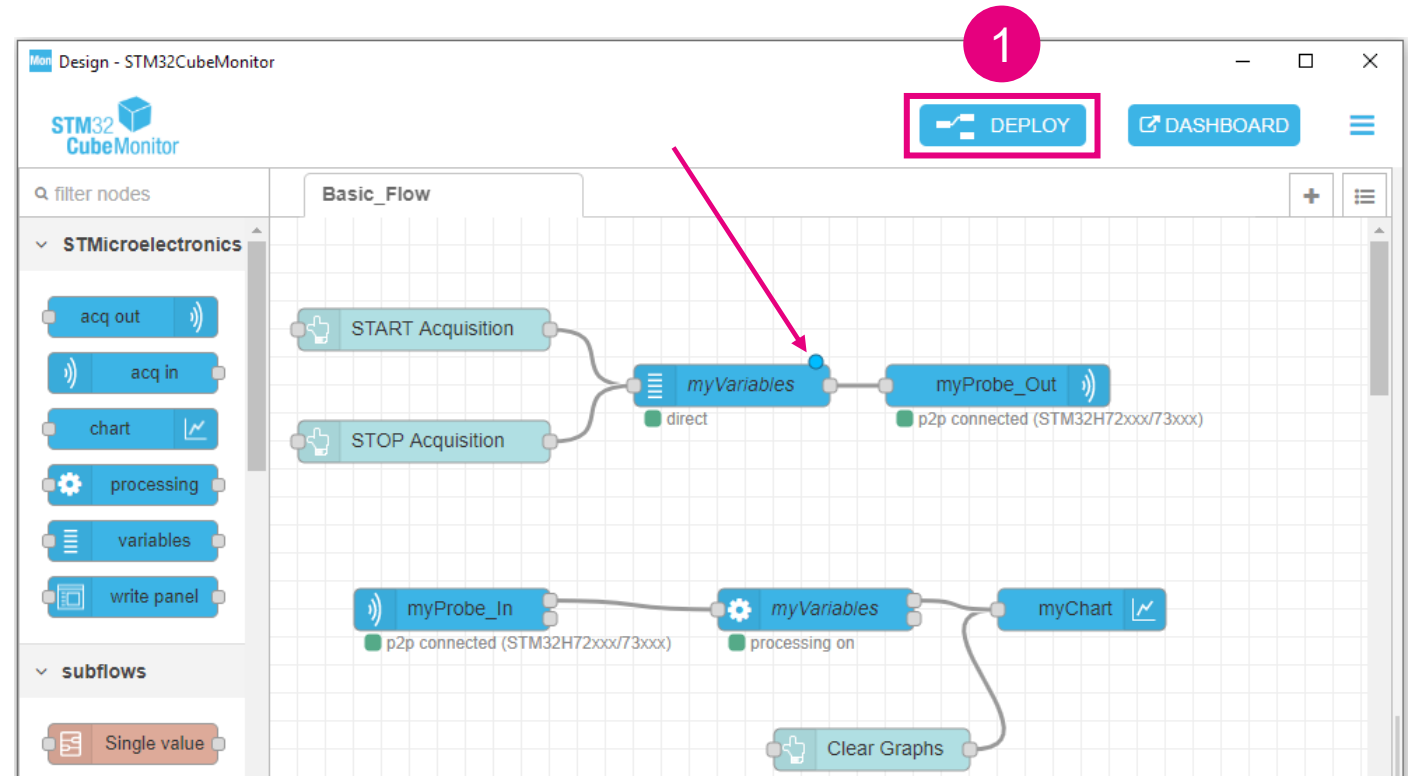
- The variable list is configured

- Name
 - basicCounter
- Address
 - 0x2400803C
- Type
 - Unsigned 16-bit

1. Click on “Done”

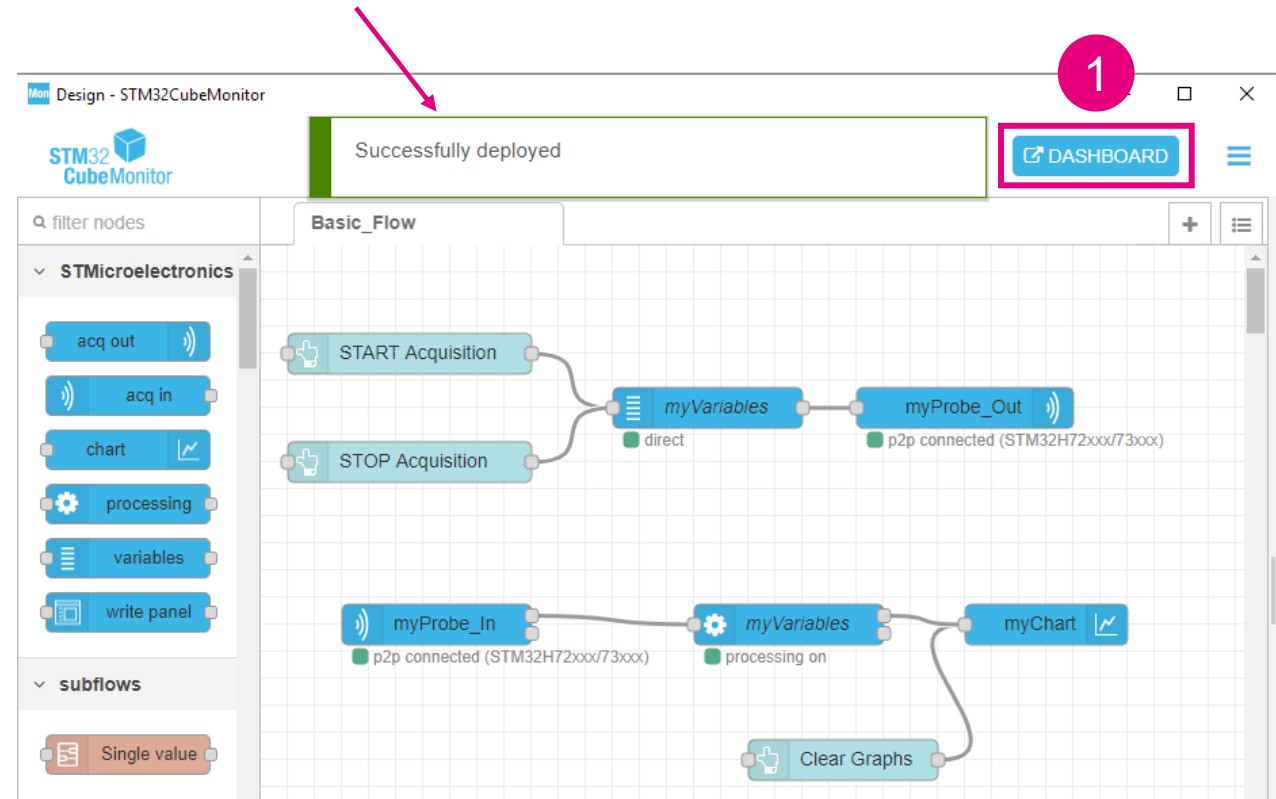


- Save the flow
 - A blue point on “myVariables” shows that the configuration has been modified.
- 1. Click on “Deploy”



Monitor and visualize read in MCU RAM memory

- Launch the dashboard
 - “Successfully deployed” is displayed
- 1. Click on “Dashboard”



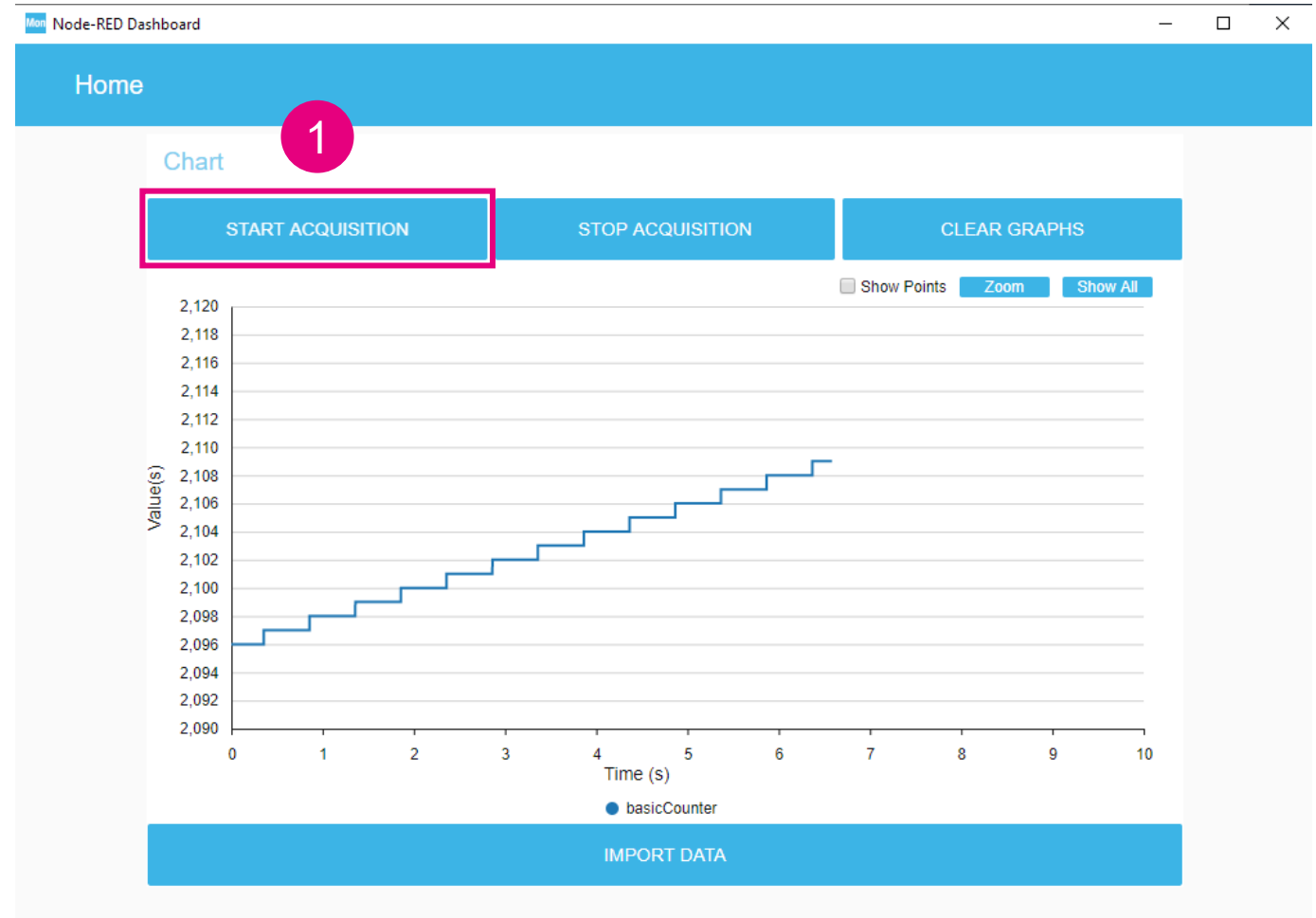
Monitor and visualize read in MCU RAM memory

- Launch the acquisition

1. Click on “START ACQUISITION”

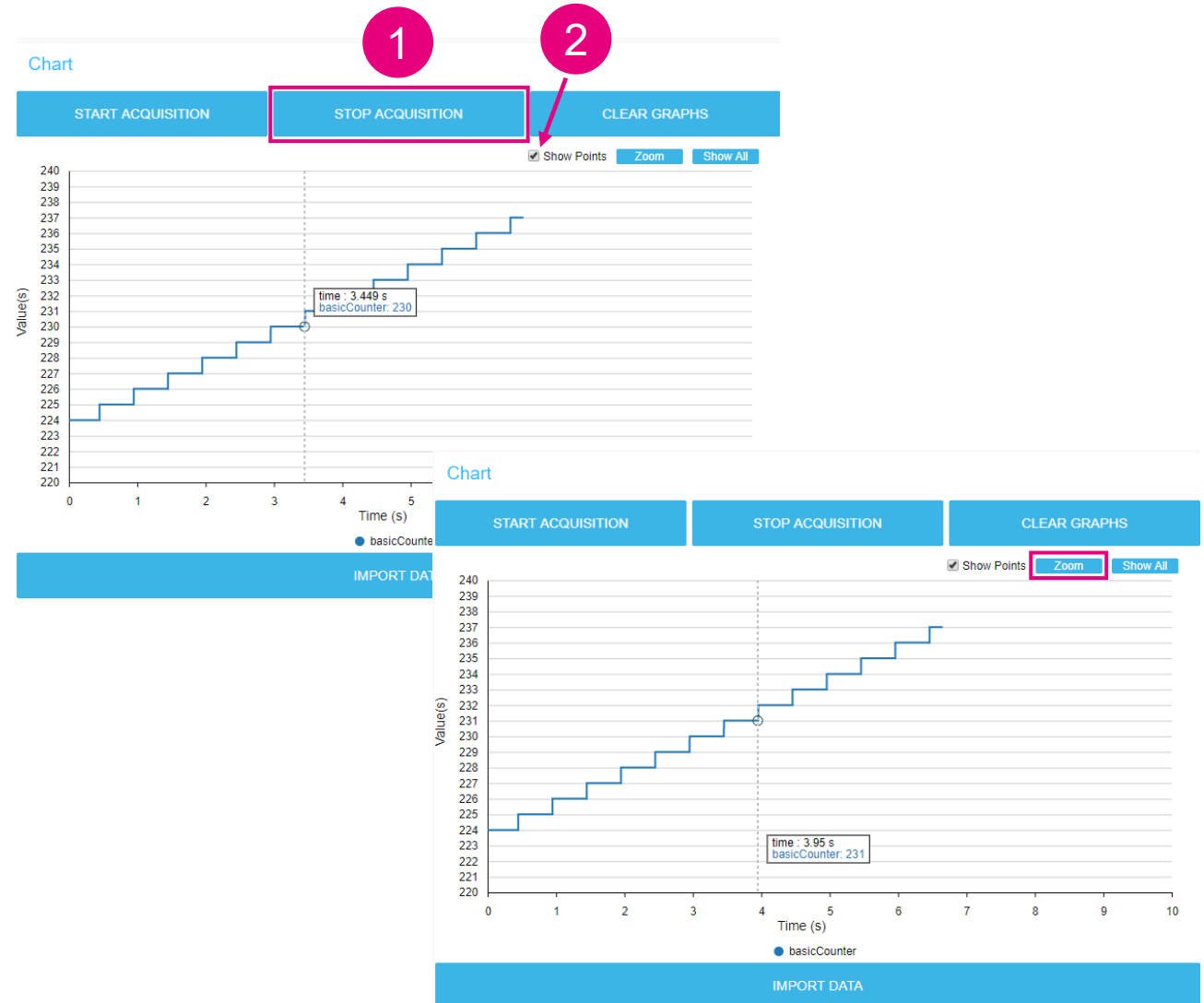
- “basicCounter” is sequentially read from the MCU RAM memory

- “basicCounter” is regularly incremented



Monitor and visualize read in MCU RAM memory

- Stop reading the variable
 1. Click on “STOP ACQUISITION”
- The times and values can be visualized
 2. Click the “Show Points” checkbox
 - ‘basicCounter’ is incremented by 1
 - “basicCounter” is incremented every 500ms
- Use the zoom for more precision
 - Zoom or Brush is selectable.
 - In zoom mode, it's possible to zoom on the curve using the mouse to select a zone.

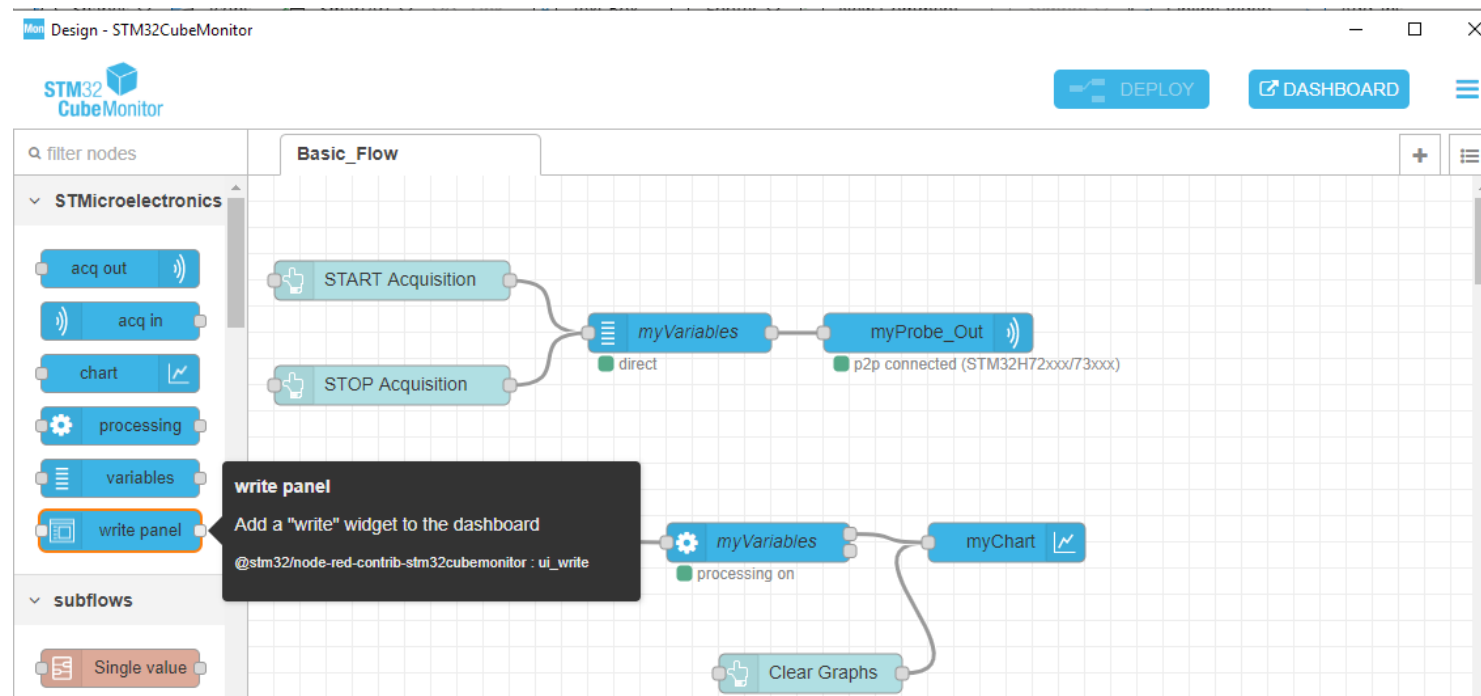


Write variables to MCU RAM memory

STM32 
CubeMonitor

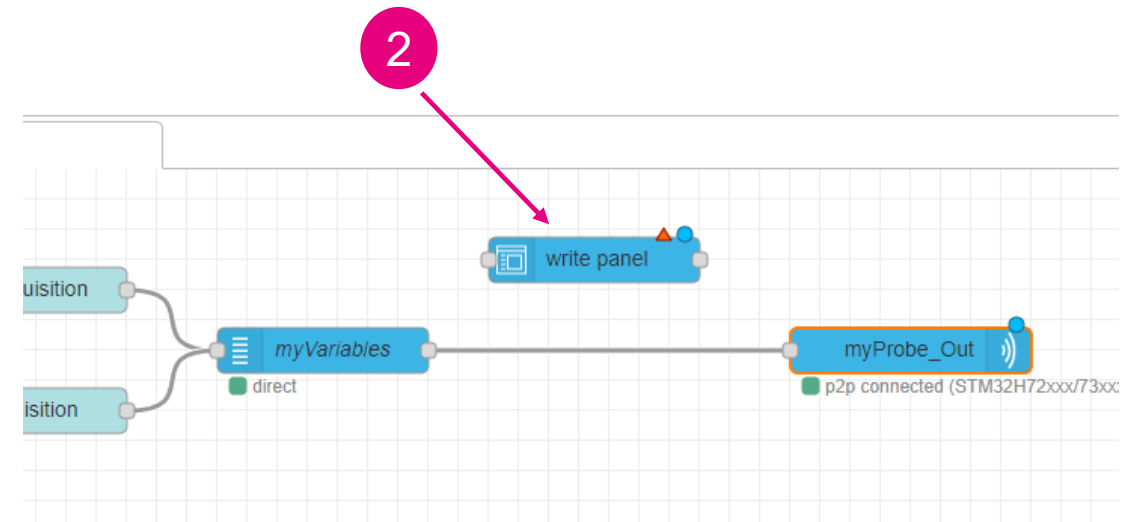
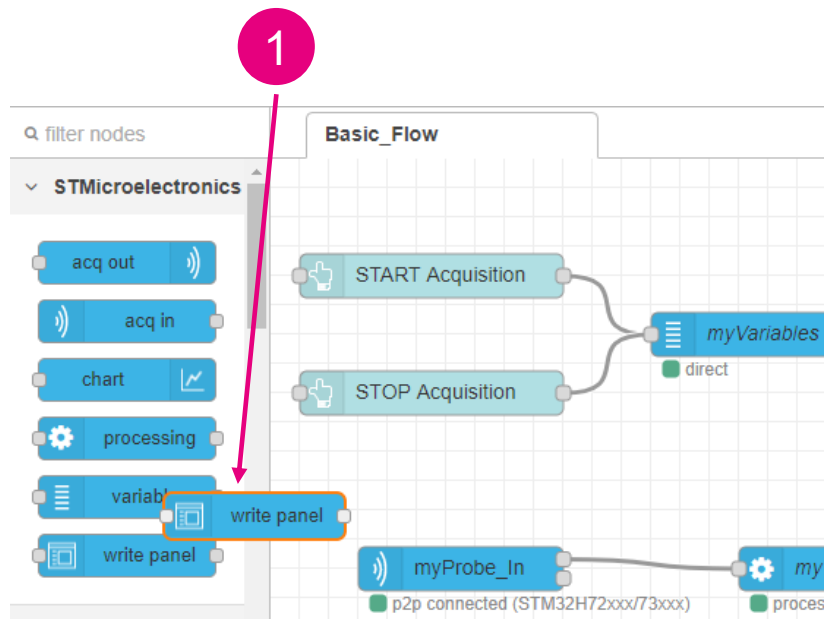
Add a widget

- Currently, no way to write a value to a variable.
 - The “write panel” widget in the node palette will be used



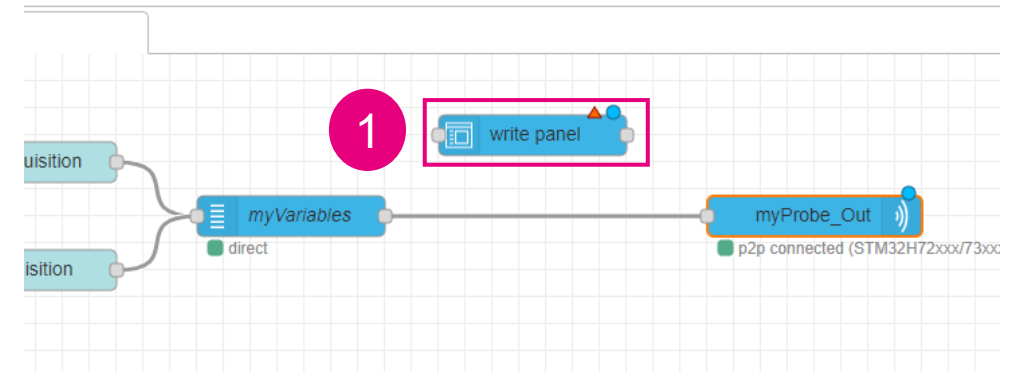
Add a widget

- Drag & drop the “write panel” in the flow editor
 1. Click on the “write panel” node and hold the button down
 2. Drag the “write panel” node to the flow editor
 - “myprobe_Out” can be dragged a bit further



Add a widget

- Set the “write panel” properties
 - A red triangle is visible on the “write panel” node
 - Properties are not configured
 - The “write panel” widget will not be added to the dashboard!

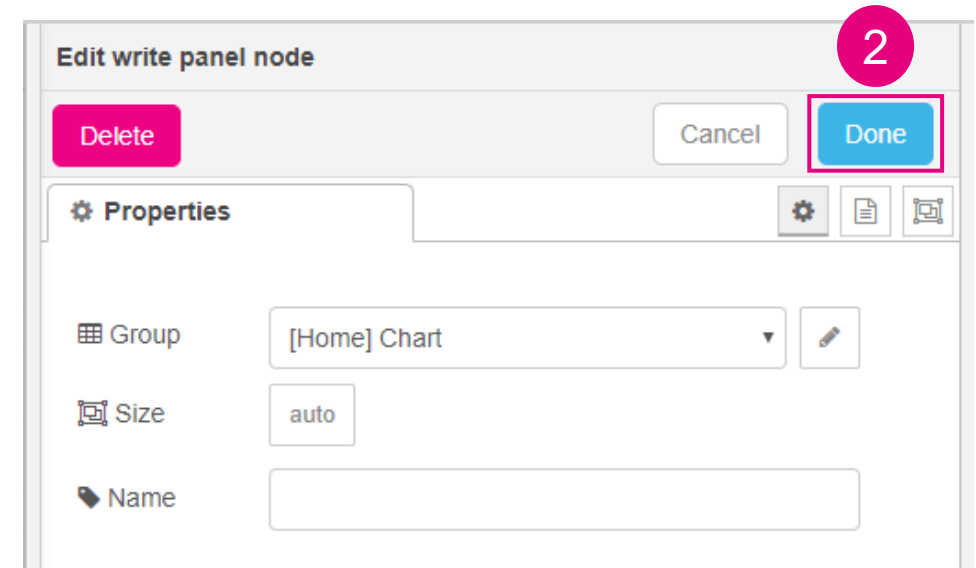


1. Double-click on the “write panel”

- “Group” has been set automatically as only one exists
 - UI group on the dashboard = Chart
 - Tab = Home
- Several groups can be created on the dashboard
- Several flows can be opened in the flow editor
 - The dashboard will then have several tabs with several UIs

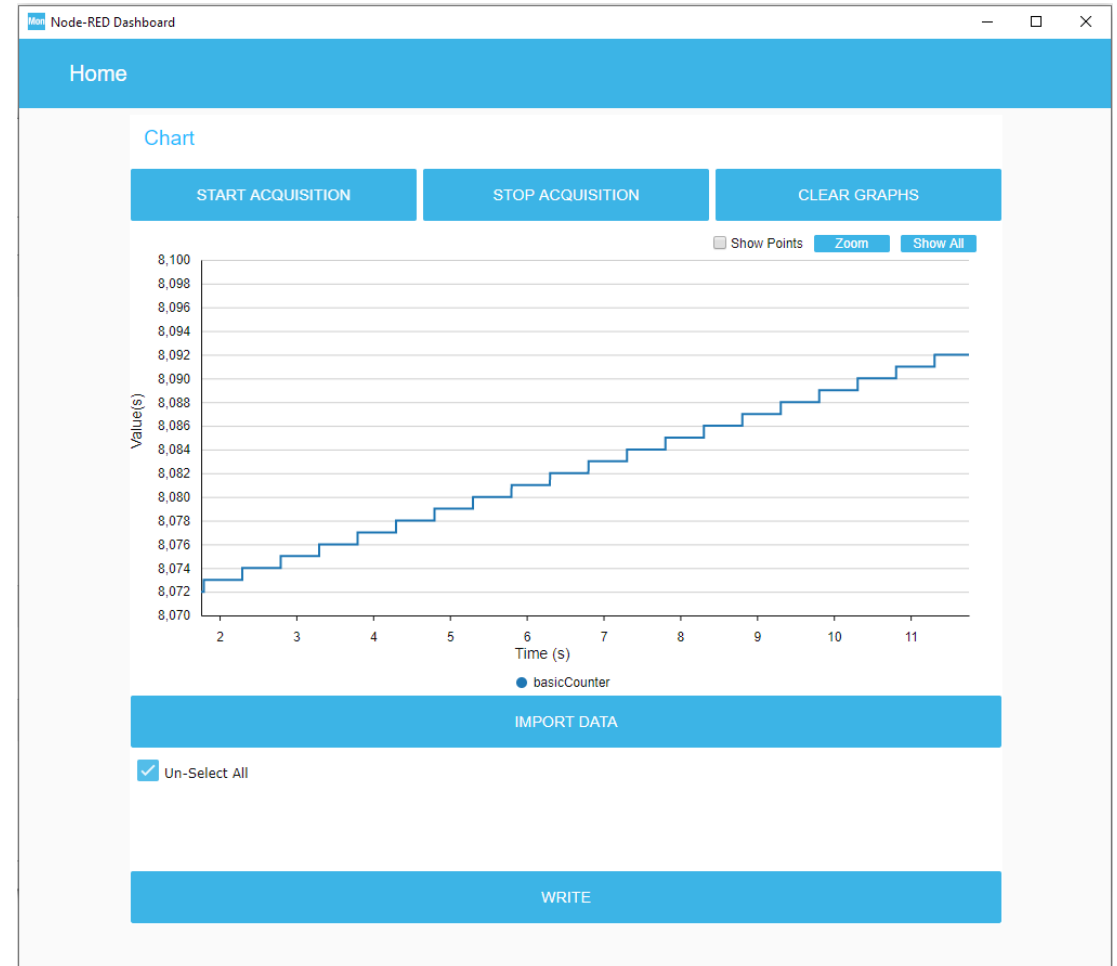
2. Click on “Done”

- The name is optional



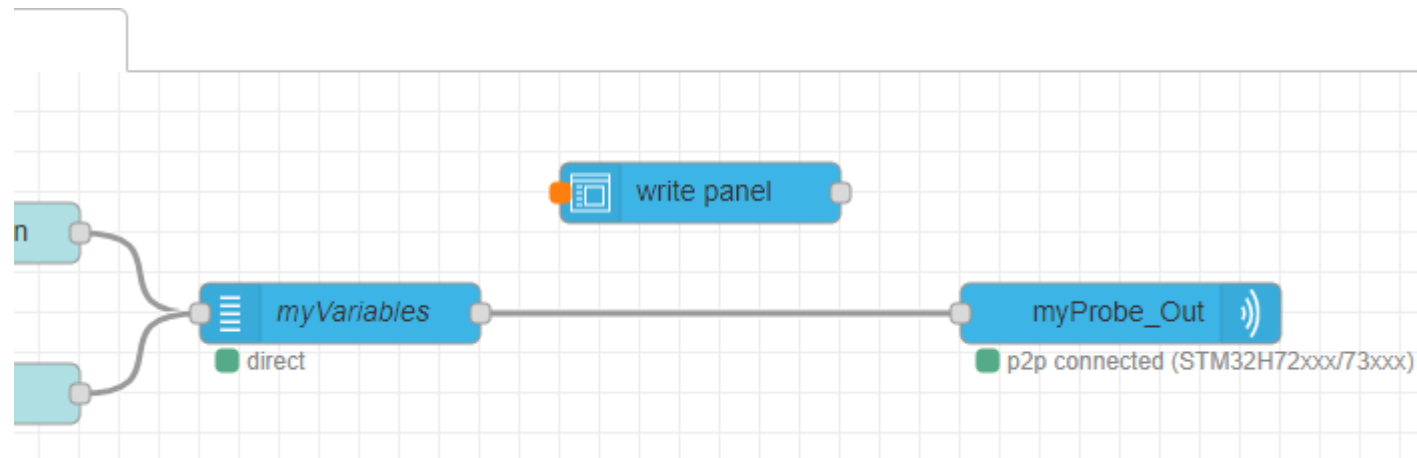
Add a widget – done!

- Launch the dashboard
 1. Click on “DEPLOY”
 2. Click on “Dashboard”
- The write panel widget has been added
 - Visual programming
 - A drag & drop and some clicks
- Let's add a variable in the panel



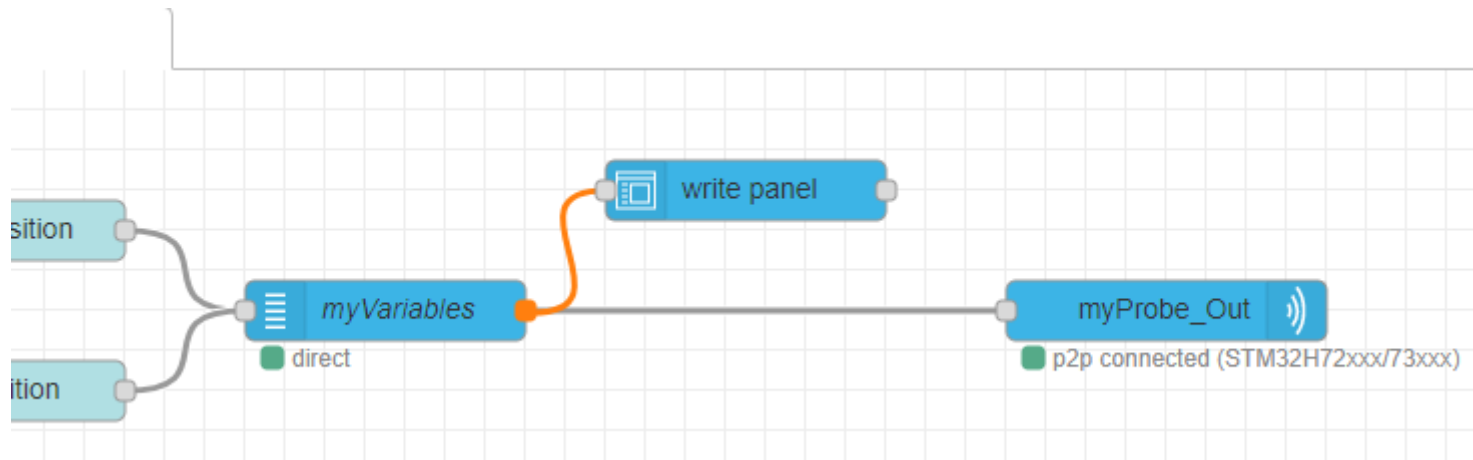
Add a variable

- Add a link with “myVariables” to populate the write panel
 1. Put the mouse on the input of the “write panel” node
 - The input point is displayed in orange



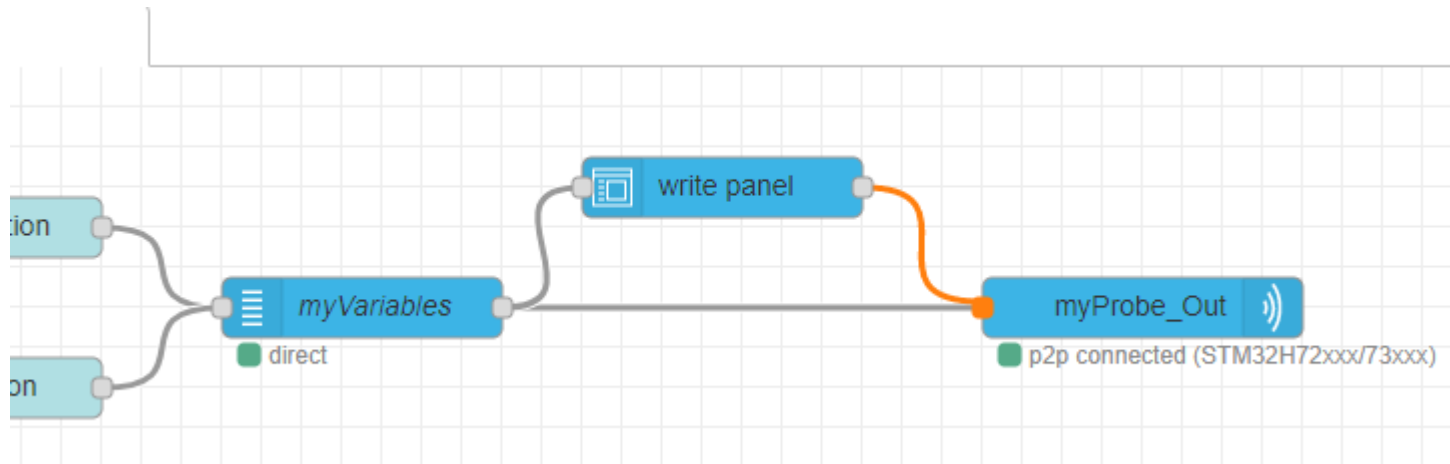
Add a variable

- Add a link with “myVariables” to populate the write panel
 2. Click and drag to the output of “myVariable”
 - Hold the mouse button down until the output point is displayed in orange
 - The link is done
 - The list of variables will appear in the write panel as soon as the “START ACQUISITION” button is clicked



Activate the “WRITE” button

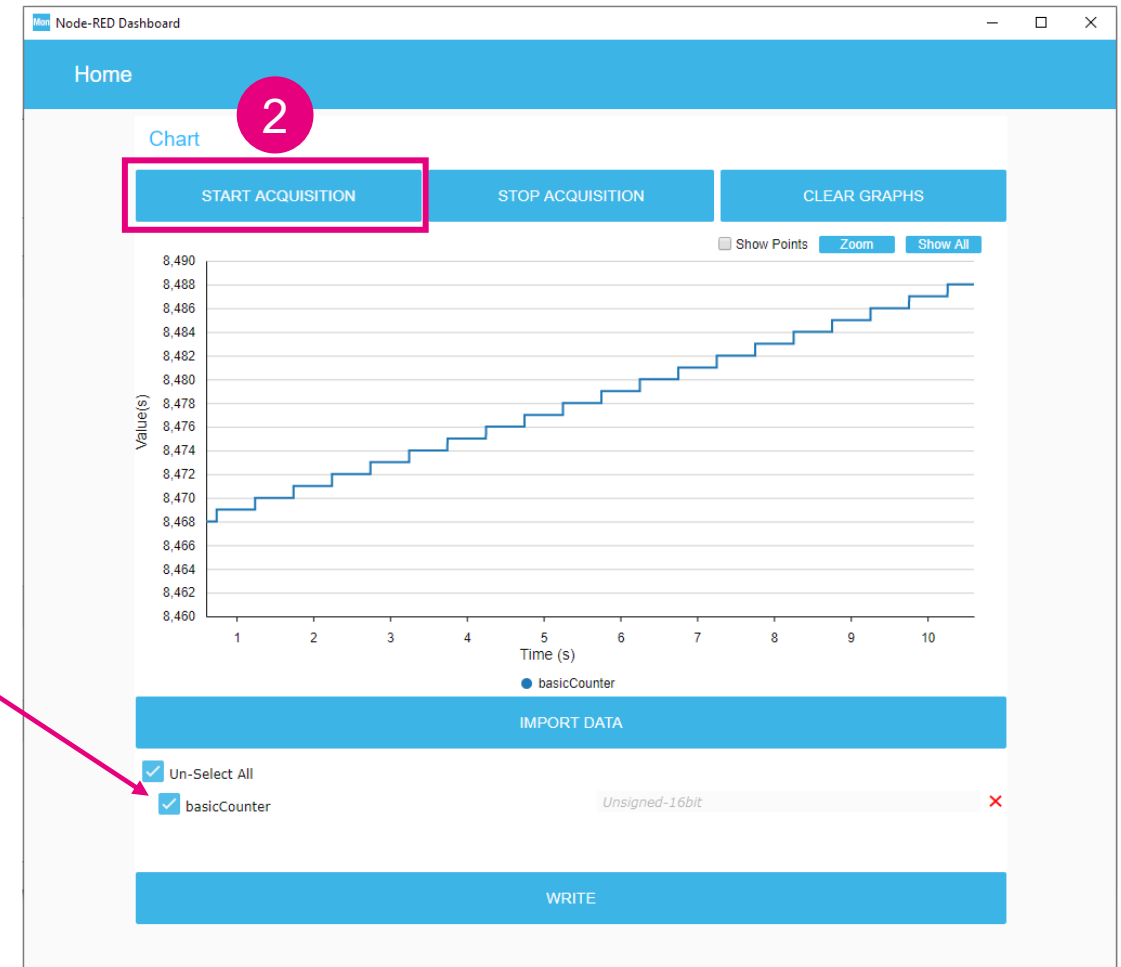
- Add a link with “myProbe_Out”
 1. Drag a link between the output of the “write panel” and the input of “myProbe_Out”
 - The link is done
 - A click on the “WRITE” button will produce a message sent on the ST-LINK:
 - “Write the selected variables”



Monitor and visualize write in MCU RAM memory

- Launch the acquisition
 1. Click on “DEPLOY”, then “DASHBOARD”
 2. Click on “START ACQUISITION”

➤ “basicCounter” is in the write panel list



Monitor and visualize write in MCU RAM memory

- Write to the basicCounter variable

1. Write a value in the text box beside “basicCounter” name
2. Click on “WRITE”

➤ “basicCounter” value is changed to the written number.



Read or Write a peripheral register

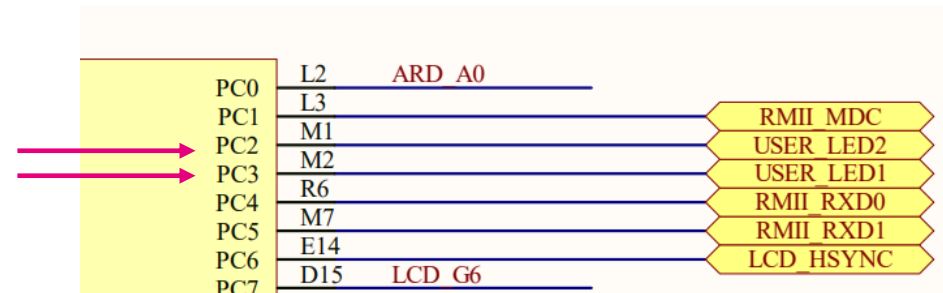
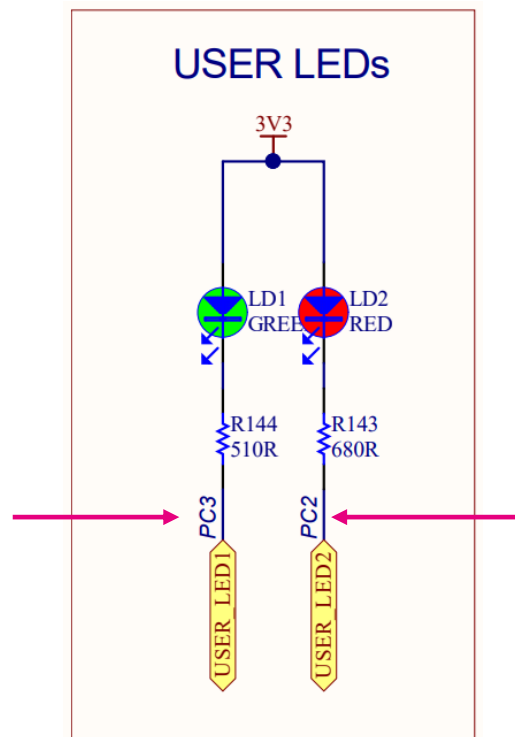
STM32 
CubeMonitor

Read/Write to an address

- **All the MCU address** space is accessible.
- But some addresses don't have a variable name in the elf file.
 - Registers
- How to read/write an address without a name?
 - Let's take the example of an I/O port

From MB1520 schematics

- The User LEDs are on GPIOC bit 2 and bit 3
- The user LEDs are ON when the I/O pin is 0



- The GPIOC base address is 0x58020800
- The ODR register has the offset 0x14
 - The lowest byte of ODR register is at 0x58020814

11.4.6 GPIO port output data register (GPIOx_ODR) (x = A to H, J, K)

Address offset: 0x14

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Table 7. Register boundary addresses⁽¹⁾

Boundary address	Peripheral	Bus	Register map
0x58026400 - 0x580267FF	HSEM	AHB4 (D3)	Section 10.4: HSEM registers
0x58026000 - 0x580263FF	ADC3		Section 28.7: ADC common registers
0x58025800 - 0x58025BFF	DMAMUX2		Section 17.6: DMAMUX registers
0x58025400 - 0x580257FF	BDMA		Section 16.6: BDMA registers
0x58024C00 - 0x58024FFF	CRC		Section 21.4: CRC registers
0x58024800 - 0x58024BFF	PWR		Section 6.8: PWR registers
0x58024400 - 0x580247FF	RCC		Section 8.7: RCC registers
0x58022800 - 0x58022BFF	GPIOK		Section 11.4: GPIO registers
0x58022400 - 0x580227FF	GPIOJ		Section 11.4: GPIO registers
0x58021C00 - 0x58021FFF	GPIOH		Section 11.4: GPIO registers
0x58021800 - 0x58021BFF	GPIOG		Section 11.4: GPIO registers
0x58021400 - 0x580217FF	GPIOF		Section 11.4: GPIO registers
0x58021000 - 0x580213FF	GPIOE		Section 11.4: GPIO registers
0x58020C00 - 0x58020FFF	PIOD		Section 11.4: GPIO registers
0x58020800 - 0x58020BFF	GPIOC		Section 11.4: GPIO registers
0x58020400 - 0x580207FF	GPIOB		Section 11.4: GPIO registers
0x58020000 - 0x580203FF	GPIOA		Section 11.4: GPIO registers

GPIOC ODR register values

- The user LEDs are ON when the I/O is 0

Green LED PC3	Red LED PC2	ODR (bin)	ODR (hex)	ODR (dec)
OFF	OFF	0b1100	0xC	12
ON	OFF	0b0100	0x4	4
OFF	ON	0b1000	0x8	8
ON	ON	0b0000	0x0	0

Add the address

- Add the address in the variable list

1. Open “myVariables” properties
2. Click on “Add custom variable”
3. Fill the fields
 - Name
 - GPIOC_ODR
 - Address
 - 0x58020814
 - Type
 - Unsigned 8-bit
4. Click on “Done”

- The name is mandatory
 - For humans
 - For STM32CubeMonitor software

The screenshot shows the STM32CubeMonitor interface. On the left, a node labeled 'myVariables' is highlighted with a red box and a red circle with the number 1. Below it, another node labeled 'myVariables' is shown with a gear icon and the text 'processing on'. On the right, the 'Edit variables node' dialog is open. The 'Delete' button is highlighted with a red box and a red circle with the number 4. The 'Done' button is highlighted with a red box and a red circle with the number 4. The 'Properties' tab is selected. The 'Group Name' is 'myVariables'. The 'Access point' is '0'. The 'Executable' is 'myVarConfig (STM32H735G-DK.elf)'. The 'Variable list' table is shown with the following data:

Name	Start Address	Type
basicCounter	0x2400802c	Unsigned 16-bit
GPIOC_ODR	0x58020814	Unsigned 8-bit

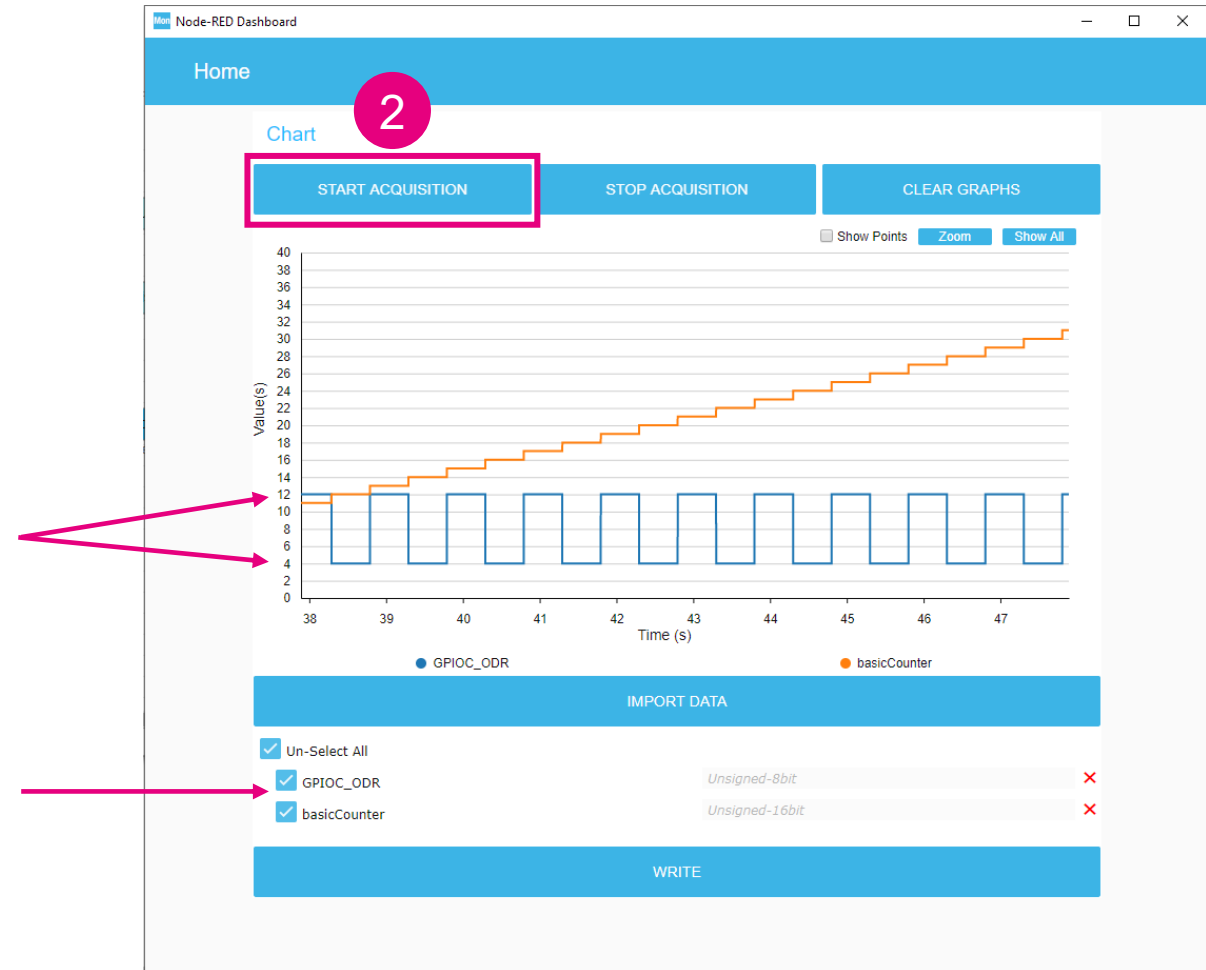
The 'GPIOC_ODR' row is highlighted with a red box and a red circle with the number 3. The '+ Add custom variable' button is highlighted with a red box and a red circle with the number 2. The 'Remove all custom variables' button is also visible.

Monitor and visualize read in MCU register fixed address

- Launch the acquisition

1. Click on “DEPLOY”, then “DASHBOARD”
2. Click on “START ACQUISITION”

- “basicCounter” is displayed
- The GPIOC_ODR is also displayed
 - It toggles between 4 and 12
 - The red LED is OFF, the green LED is toggling with a period of 1s (twice the period of “basicCounter”)



Monitor and visualize write in MCU register fixed address

- Light on the red LED

1. Click on basicCounter name under the chart
 - The “basicCounter” curve disappear.
 - The vertical scale is resized
 2. Write 0x0 in the text box beside “GPIOC_ODR”
 3. Click on “WRITE”
- The GPIOC_ODR values have changed
 - It toggles between 0 and 8
 - The red LED is ON, the green LED is toggling



Monitor and visualize write in MCU register fixed address

- Light off the red LED

1. Write 0x04 in the text box beside "GPIOC_ODR"
2. Click on "WRITE"

- The GPIOC_ODR values have changed again
 - It toggles between 4 and 12
 - The red LED is OFF again



We have seen...

- How to **configure** the flow
 - How to connect to the STM32 MCU board through ST-Link
 - How to get the variable names from the executable file
- How to **access** to the STM32 memory
 - Read / Write using the variable name
 - Read / Write using the address
 - All the STM32 address space is accessible
- How to add a **widget** to the dashboard
 - Drag & drop on the flow editor
 - Insert in the flow
 - Make the connection with previous and next node

STMH72x/3x Demo flow

STM32 
CubeMonitor

STMH72x/3X Demo flow

- An example of customization
 - More widgets
 - But same concepts
- Let's import the flow

Disable a flow

- Disable the “Basic_Flow”

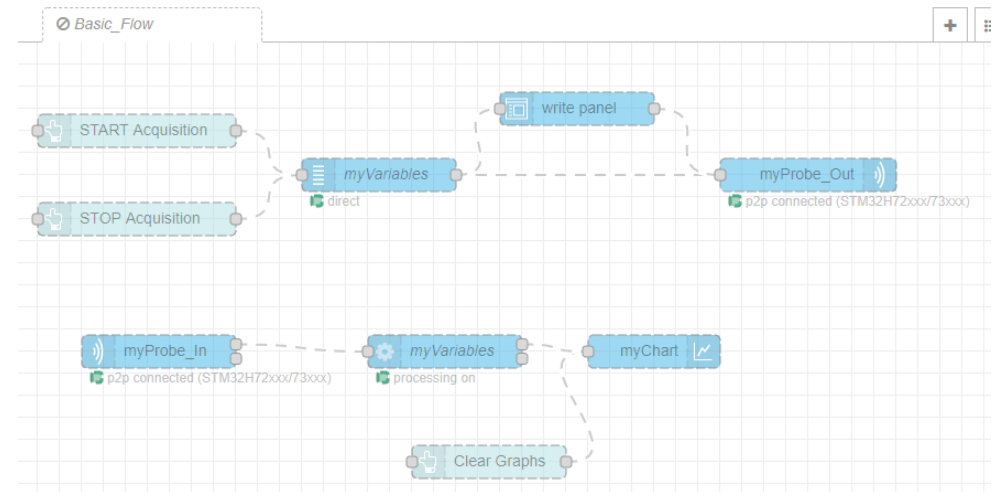
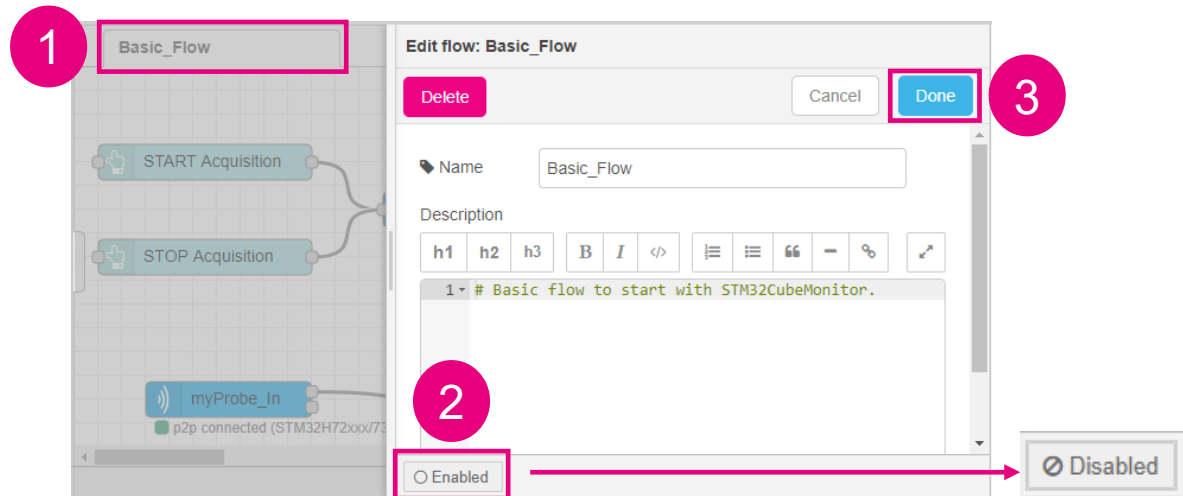
1. Double-click on the “Basic_Flow” tab

2. Click in Enabled

- It is changed to disabled

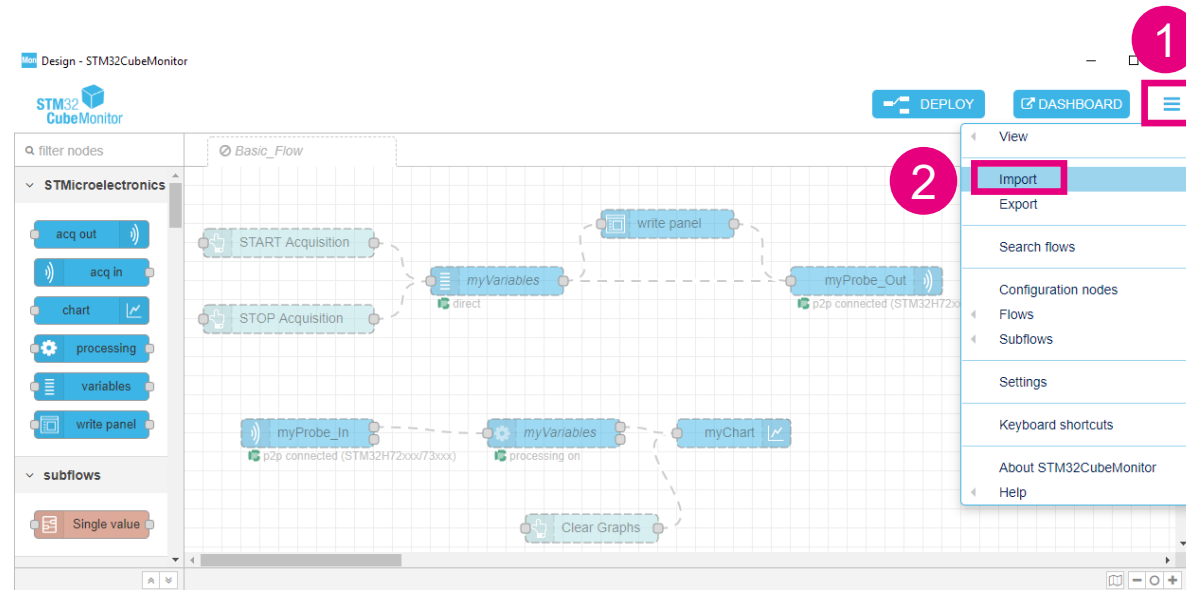
3. Click on Done

- The “Basic_Flow” tab remains in the flow editor but no “Home” tab will be accessible in the dashboard.



Import a flow

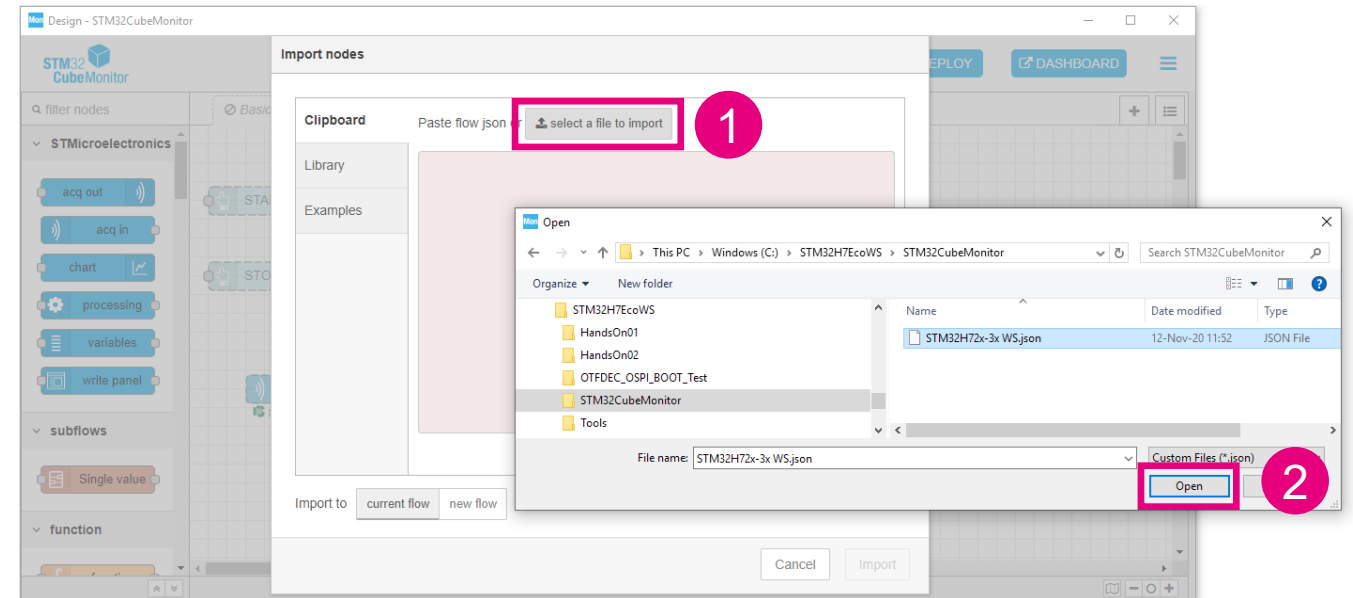
- **Please click on “DEPLOY”**
 - If not closed and acquisition not stopped, the dashboard is disabled and the ST-Link is relaxed.
- Open the menu and select import
 1. Click on the menu
 2. Click on “Import”



Import a flow

- Import from a file

1. Click on “select a file to import”
2. Open the STMH72x/3X Demo flow
 - In C:\STM32H7EcoWS\STM32CubeMonitor
 - STM32H72x-3x WS.json



Import a flow

- Import from a file

1. Click on “Import”

Import nodes

Clipboard Paste flow json or [select a file to import](#)

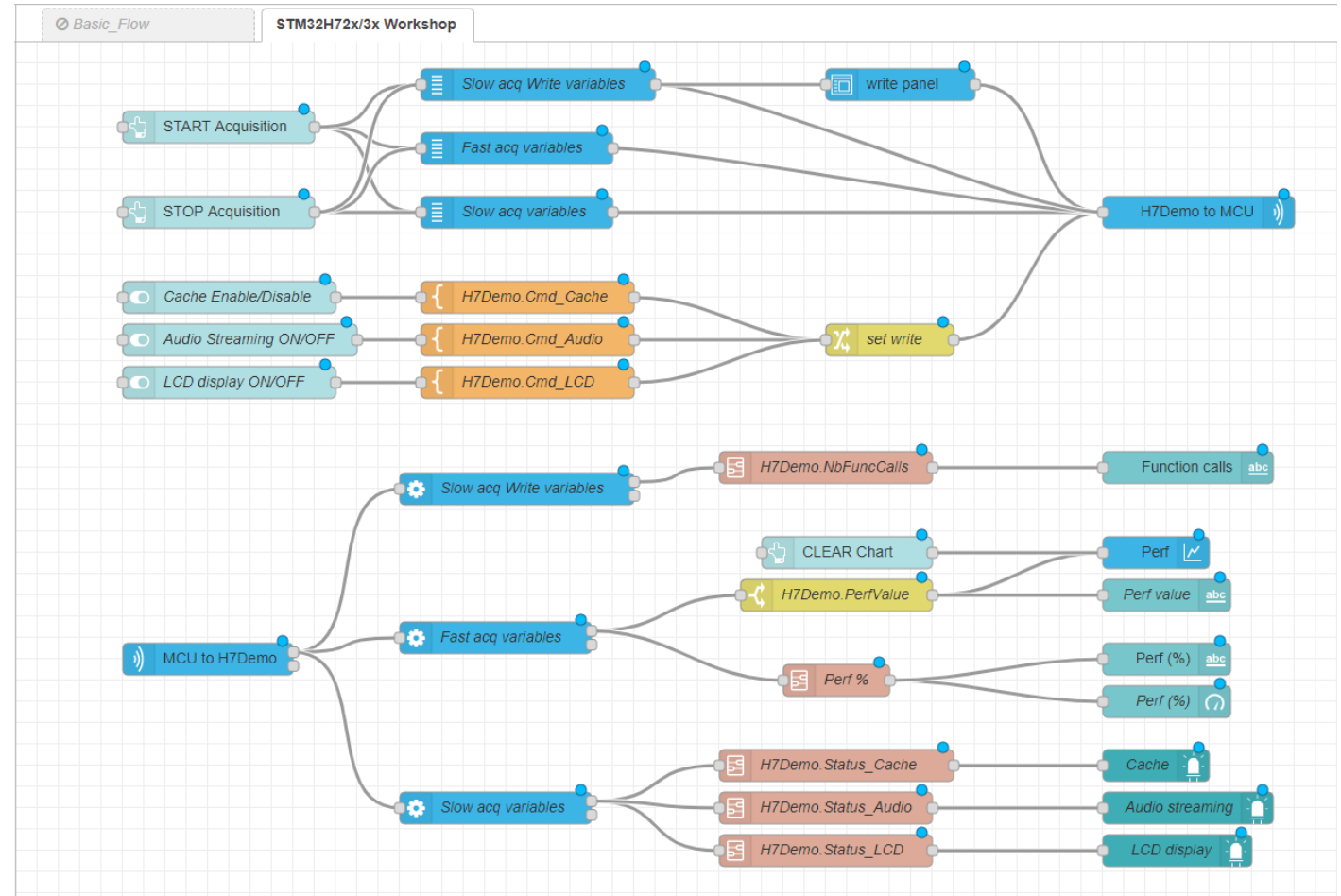
Library	<pre>[{ "id": "59680ee2.a50cd", "type": "subflow", "name": "Single value", "info": "The 'single value' subflow allows to :\r\n * Filter Data to extract only one variable after the \"processing node\" step .\r\n * Modify the message to fit with standard widgets such as gauges.\r\n * Limit the number of messages to a maximum of 10 msg per second.\r\n\r\n### **Input**\r\n\r\nThe subflow 'single value' takes in input the 'msg.payload'\r\nfrom the processing output. All the messages sent by the processing node have the same structure.\r\n\r\n \"payload\": {\r\n \r\n \"groupname\": \"Snapshot2\", \r\n \"variabledata\": [\r\n \r\n {\r\n \"y\": \"9\", \r\n \"x\": \r\n \"1567509421459\", \r\n }, \r\n {\r\n \"y\": \"9\", \r\n \"x\": \"1567509421459\", \r\n }, \r\n {\r\n \"y\": \"9\", \r\n \"x\": \"1567509421459\", \r\n }] }]</pre>
Examples	

Import to

1

Import a flow

- The STMH72x/3X Demo flow is now visible in the flow editor
- More nodes, more links, but...



Same flow structure

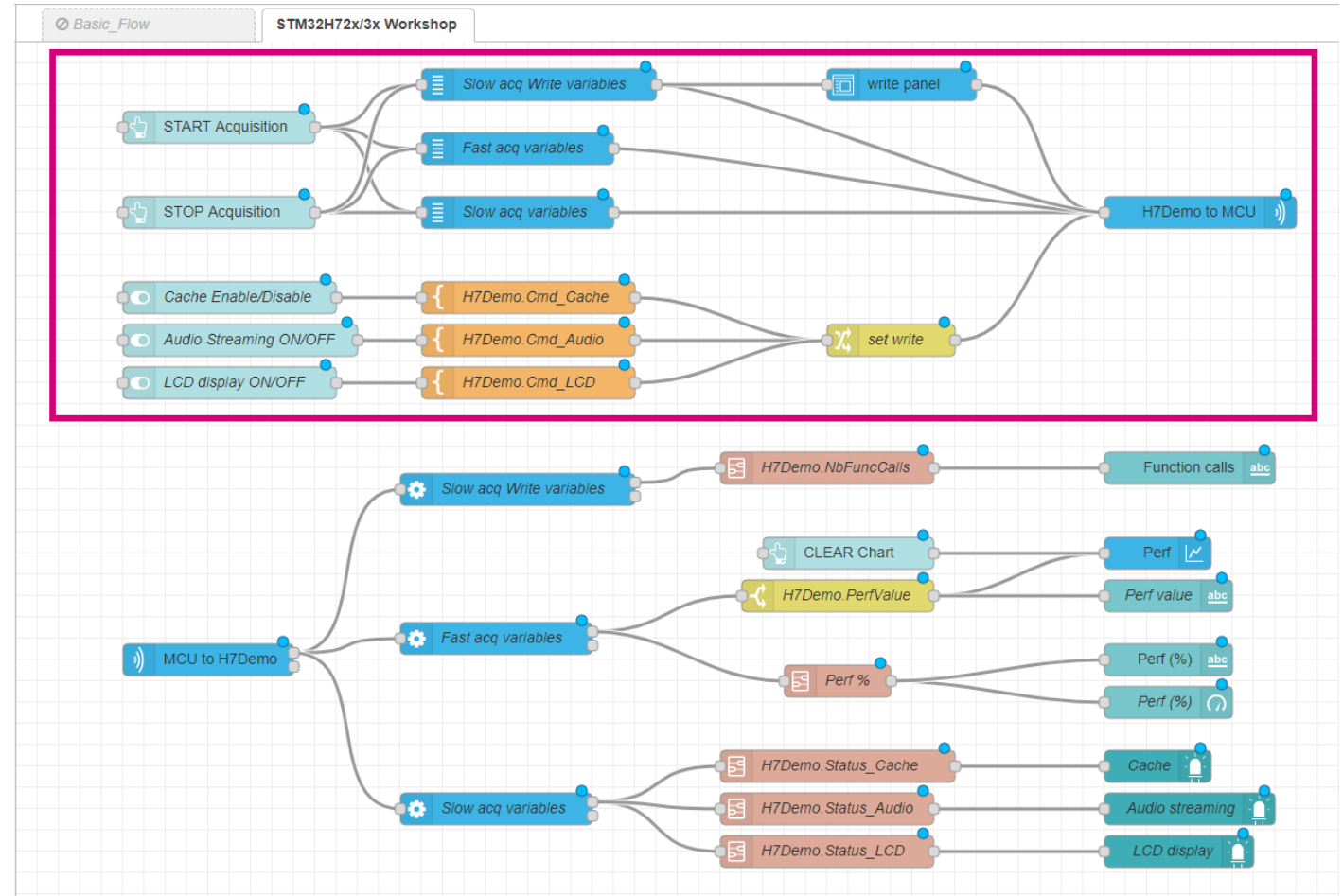
- One TX path from PC to probe

- But 3 variable lists

- Only 1 variable can be written in the write panel
- The sampling frequency is slow or fast

- And a new way to write in MCU RAM memory

- 3 switch widgets allows to write to 3 variables
- But the same message to the ST-Link probe than the write panel
 - “write this value at that address”
 - 0 for switch OFF, 1 for switch ON



Same flow structure

- One RX path from probe to PC

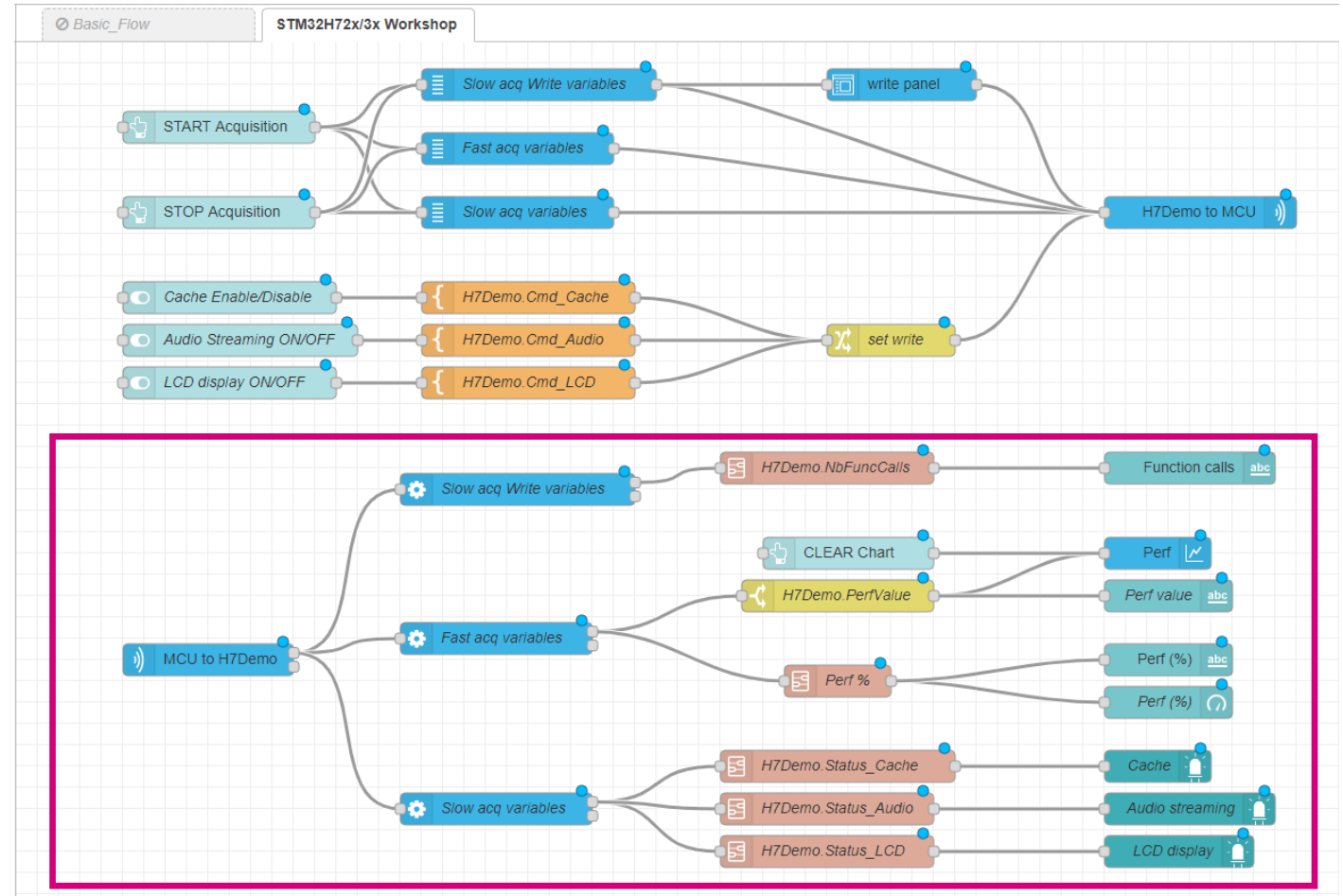
- 3 variable processing nodes
 - Paired with each variable list

- Again 1 chart but also other output widgets
 - Text, gauge, LEDs

- In between, selection nodes
 - Select which variable is displayed on which widget
 - A variable can be displayed on several widgets
 - Several variables can be displayed on a widget

- **More customization but...**

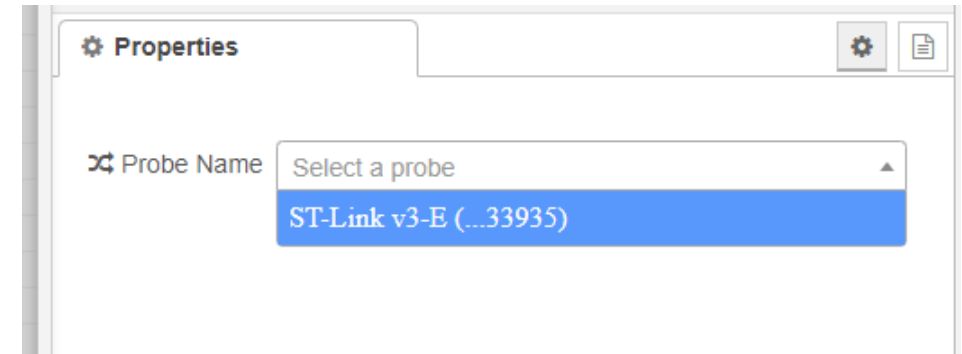
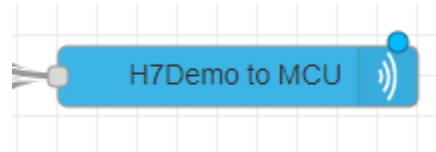
- **Nothing more difficult** than "Basic_Flow"



Prepare the flow

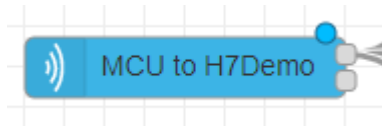
- Set ST-Link (Monitor to MCU)

1. Double-click on “H7Demo to MCU”
2. Click on the pen of “Probe Config”
3. Select the ST-Link probe
4. Click on “Add”
5. Click on “Done”



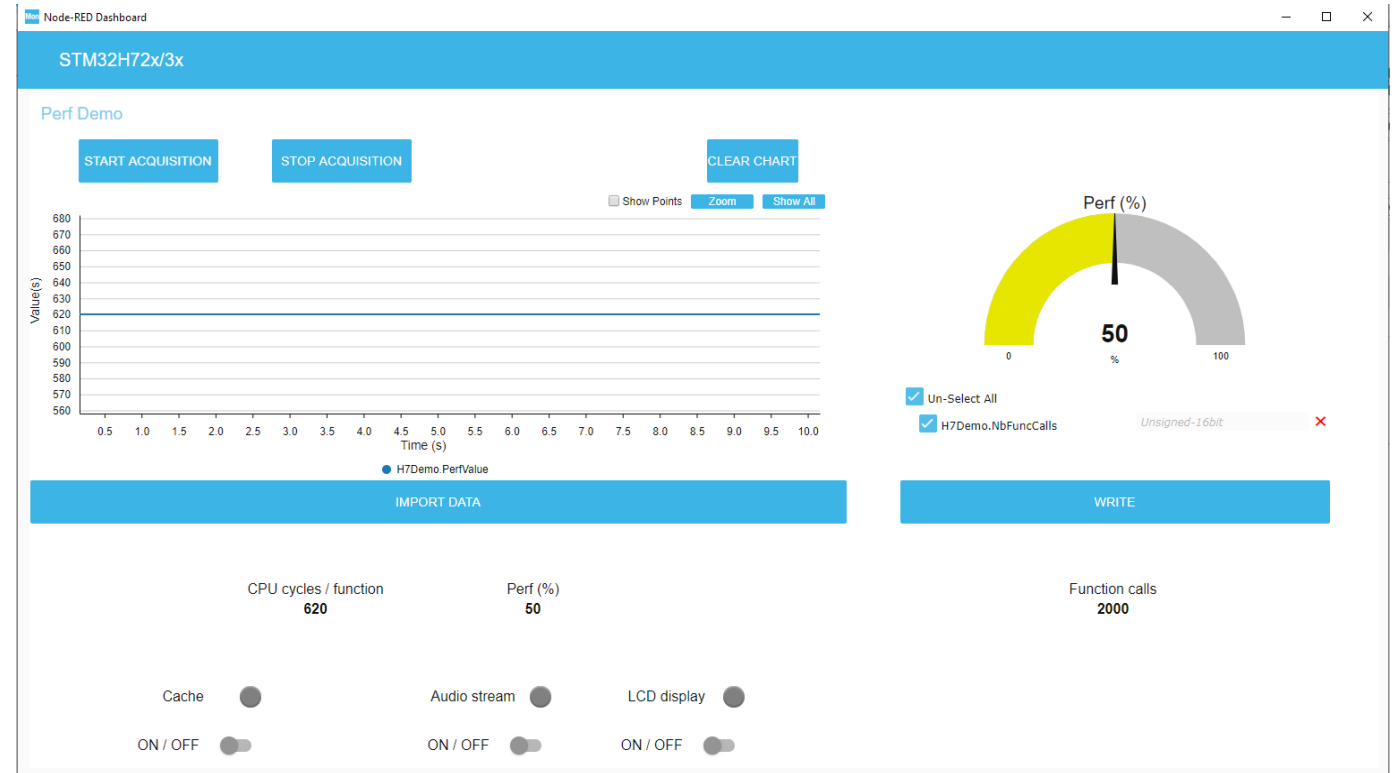
- Set ST-Link (MCU to Monitor)

- Do the same for “MCU to H7Demo”



Launch the dashboard

- The dashboard displays
 - Known widgets
 - 3 Buttons
 - START, STOP, CLEAR
 - 1 chart, 1 write panel
 - New widgets
 - 1 gauge
 - 3 text fields
 - 3 switches and LEDs



Variables at fixed address

- Other firmware will be downloaded to the board for HandsOn2 and HandsOn3
- But the same variables will be used.
 - Variables in a structure at fixed address
 - H7Demo - 0x24000000 to 0x24000008
 - Reads and writes at the same addresses
 - No need to update the elf file

The screenshot displays the STM32H72x/3x Workshop interface. On the left, a flowchart shows nodes for 'START Acquisition', 'STOP Acquisition', 'Cache Enable/Disable', 'Audio Streaming ON/OFF', and 'LCD display ON/OFF'. These nodes are connected to 'Slow acq Write variables', 'Fast acq variables', and 'Slow acq variables' blocks. A 'MCU to H7Demo' block is also present, connected to the 'Fast acq variables' and 'Slow acq variables' blocks. The right panel shows the 'Edit variables node > Edit exe-config node' dialog. It includes fields for 'Name' (MySlowVarList), 'Folder' (C:\STM32H7EcoWS\HandsOn01), and 'File' (STM32H735G-DK.elf). Below these fields is a table titled 'Variable List' with columns 'Select Name', 'Start Address', and 'Type'. The table lists various variables, some of which are selected with checkboxes.

Select Name	Start Address	Type
☐ basicCounter	0x2400802c	Unsigned 16-bit
☐ D1CorePrescTable[0]	0x08001dfc	Unsigned 8-bit
☐ H7Demo.Cmd_Audio	0x24000001	Unsigned 8-bit
☐ H7Demo.Cmd_Cache	0x24000000	Unsigned 8-bit
☐ H7Demo.Cmd_LCD	0x24000002	Unsigned 8-bit
☐ H7Demo.NbFuncCalls	0x24000008	Unsigned 16-bit
☐ H7Demo.PerfValue	0x24000006	Unsigned 16-bit
☒ H7Demo.Status_Audio	0x24000004	Unsigned 8-bit
☒ H7Demo.Status_Cache	0x24000003	Unsigned 8-bit
☒ H7Demo.Status_LCD	0x24000005	Unsigned 8-bit
☐ SystemCoreClock	0x24008000	Unsigned 32-bit
☐ SystemD2Clock	0x24008004	Unsigned 32-bit
☐ uwTick	0x24008030	Unsigned 32-bit
☐ uwTickFreq	0x2400800c	Signed 8-bit
☐ uwTickPrio	0x24008008	Unsigned 32-bit

Launch the dashboard

- The current firmware is a simulation firmware
 - The same logic but only **dummy actions**

- Example of the cache switch

- When HandsOn2 firmware will do

```
if(H7Demo.Status_Cache != H7Demo.Cmd_Cache){  
    if (H7DEMO_ON == H7Demo.Cmd_Cache){  
        SCB_EnableICache();  
        H7Demo.Status_Cache = H7DEMO_ON;  
    }  
}
```

- The Perf varies because of cache on/off

- The current firmware does

```
PerfPercent += 50;  
H7Demo.Status_Cache = H7DEMO_ON;
```

- The cache is **always off**
 - The Perf is **dummy**



Going further - Documentation & Links

STM32 
CubeMonitor

Conclusion

- STM32CubeMonitor is a non-intrusive tool to monitor a running application
 - Uses the MCU debug bloc and the SWD protocol
- STM32CubeMonitor is easily customizable.
 - We have seen the basics:
 - How to **configure** the flow
 - How to **access** to the STM32 memory
 - How to add **widgets** to the dashboard
 - You are **ready to design your own** dashboard
- It is now ready for HandsOn2 and HandsOn3.
 - **Please, close it now** (to allow CubeIDE debug connection).
 - It will connect automatically to the target STM32 board at next start.

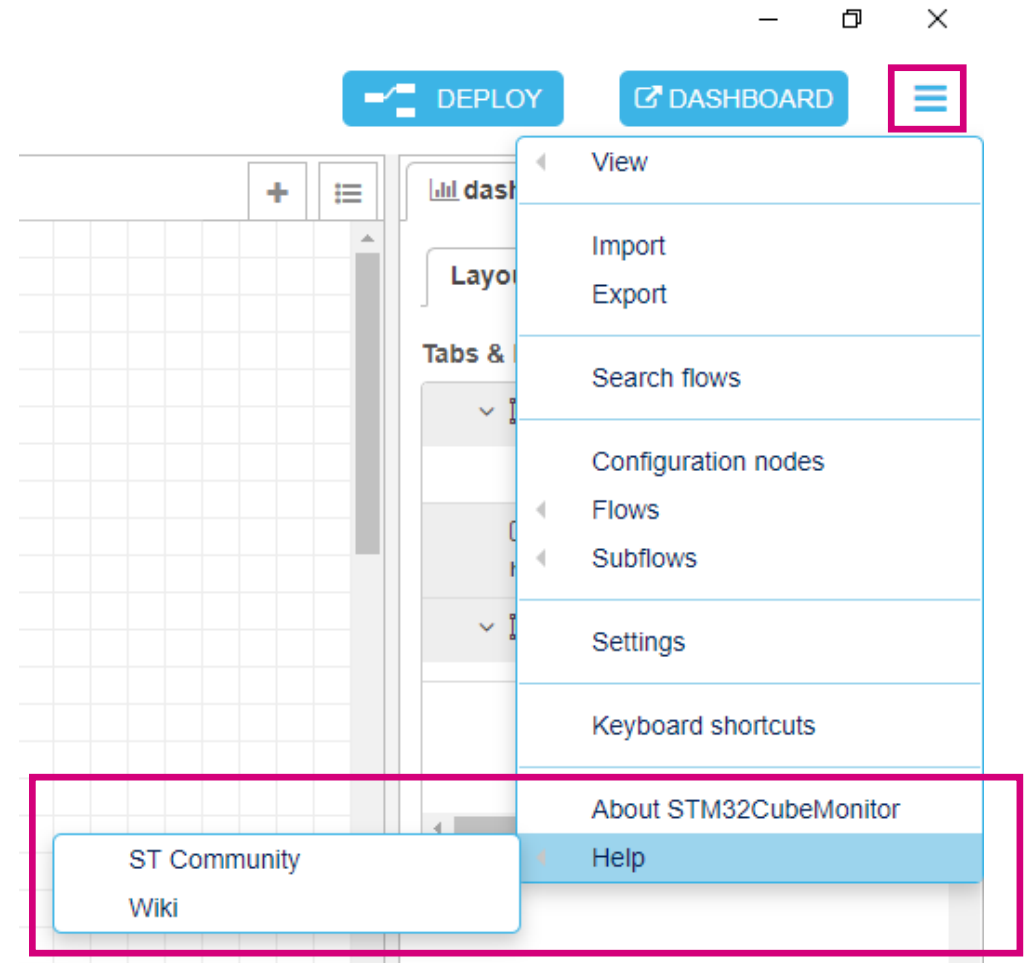
Resources and useful links

- [STM32CubeMonitor web page](#)
- [STM32CubeMonitor : Getting Started](#)
- [STM32 MCU Wiki](#)
- [Webinar Replay: Application firmware monitoring with New STM32CubeMonitor](#)
 - At the bottom of STM32CubeMonitor web page
- [Node-RED documentation](#)



To go further

- The help is directly accessible through the menu
 - Allow to find information to guide the user
 - ST Community
 - STM32 MCU wiki



Thank you

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.



life.augmented